

DEVELOPMENT AND EVALUATION OF A RISK-AWARE VISUAL INSPECTION FRAMEWORK FOR INDUSTRIAL ANOMALY DETECTION

by
Jesse Asemota

Thesis submitted to University of Plymouth
in partial fulfilment of the requirements for the degree of

MSc Robotics

University of Plymouth

Faculty of Science & Engineering

September 2026

Masters Dissertation License

This material has been deposited in the University of Plymouth Learning & Teaching repository under the terms of the student contract between the students and the Faculty of Science & Engineering.

The material may be used for internal use only to support learning and teaching.

Materials will not be published outside of the University and any breaches of this licence will be dealt with following the appropriate University policies.

Copyright Statement

This copy of the thesis has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with the author and that no quotation from the thesis and no information derived from it may be published without the author's prior written consent.

Abstract

Deep learning progressively enhances industrial visual inspection; yet numerous anomaly-detection investigations continue to prioritise model-level performance over traceable quality-control decision-making. This project developed and evaluated a risk-aware visual inspection framework for identifying industrial anomalies, using the bottle, capsule, and hazelnut categories from the MVTec Anomaly Detection dataset. PatchCore and PaDiM were implemented as the main architectures for anomaly detection, as both learn the normal product appearance from images free of defects but through distinct methodologies of normality modelling. PatchCore employs a memory-bank framework for normal patch embeddings, while PaDiM characterises normal patch features using position-specific multivariate Gaussian distributions.

The study integrated image-level anomaly detection with localisation, binary mask generation, constrained semantic interpretation, confidence fusion, deterministic risk classification, human-review gating, and auditable quality-disposition support. The implemented prototype stored inspection data and operator reviews in PostgreSQL, exposed backend functionality via FastAPI, and offered a linked NiceGUI dashboard for reviewing anomaly scores, heatmaps, masks, semantic labels, risk classes, and quality assurance recommendations.

The results indicated that PatchCore exhibited the strongest and most consistent image-level detection performance across the evaluated categories, with PatchCore using a ResNet50 backbone selected as the preferred architecture. PaDiM continued to serve as an important comparative benchmark and localisation reference, illustrating that the

efficacy of image-level detection and the quality of pixel-level masks should be evaluated separately. The main contribution of the project is not in a novel anomaly-detection model but in an integrated framework that links visual anomaly evidence to risk-aware industrial decision-making. The study illustrates how anomaly detection may be broadened beyond mere classification into a traceable, human-supervised, and operationally significant inspection process appropriate for additional validation in industrial manufacturing environments.

Declaration of Word Count

I declare that the word count for this dissertation is 12,055 words, including the abstract and excluding the contents page, references, appendices and captions, in accordance with the PROJ518 dissertation guidelines.

Table of Contents

Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Motivation of study	2
1.3 Research problem	3
1.4 Research aim	4
1.5 Research objectives	4
Chapter 2: Literature Review	6
2.1 Introduction	6
2.2 Industrial visual inspection and one-class anomaly detection	6
2.3 Theoretical framework: PatchCore and PaDiM	8
2.4 From anomaly evidence to risk-aware inspection	9
2.5 Research gap and alignment with the proposed study	10
Chapter 3: Methodology	12
3.1 Introduction	12
3.2 Methodological rationale and research design	12
3.3 Data collection rationale and data organisation	14
3.3.1 Dataset selection	14
3.3.2 Data partitioning and use of ground truth	15
3.4 Experimental design	16
3.5 Software frameworks and computational environment	17
3.6 Detection pipeline and mathematical modelling	19
3.6.1 Shared feature-extraction approach	19
3.6.2 PaDiM distribution modelling	19
3.6.3 PatchCore memory-bank modelling	20
3.7 Anomaly localisation and post-processing	22
3.8 Constrained semantic interpretation	23
3.9 Risk determination and confidence fusion	24
3.9.1 Deterministic risk classification	24

3.9.2 Confidence fusion	24
3.10 Quality disposition and audit storage	26
3.11 Operational application layer.....	27
3.12 Evaluation methodology	27
3.12.1 Image-level detection metrics	27
3.12.2 Localisation metrics	28
3.12.3 Operational evaluation	29
3.13 Experimental sequence.....	29
3.14 Reliability, validity and reproducibility	30
3.15 Ethical and governance considerations.....	31
3.16 Chapter summary	31
Chapter 4: Results and Discussion	33
4.1 Introduction	33
4.2 Overview of the implemented inspection pipeline	33
4.3 Anomaly Detection Architecture Comparison.....	34
4.3.1 Experimental configuration	34
4.3.2 PatchCore performance.....	35
4.3.3 PaDiM performance	36
4.3.4 Cross-category interpretation.....	36
4.4 Selection of the preferred detection architecture.....	37
4.5 Integrated image-level detection results.....	38
4.5.1 Bottle results	38
4.5.2 Hazelnut results	40
4.5.3 Cross-category operational interpretation	40
4.6 Localisation Performance.....	40
4.6.1 Bottle localisation	41
4.6.2 Hazelnut localisation	41
4.6.3 Capsule localisation	41
4.6.4 Comparison of image-level and pixel-level performance	42
4.7 Localisation optimisation threshold	42
4.7.1 Bottle threshold optimisation	43
4.7.2 Hazelnut threshold optimisation	43

4.7.3 Capsule threshold optimisation.....	43
4.7.4 Implications of threshold tuning	43
4.8 Semantic interpretation and review workload.....	44
4.9 Risk-governed quality disposition.....	45
4.10 PostgreSQL audit database validation	46
4.11 FastAPI and NiceGUI operational validation	48
4.11.1 FastAPI backend	48
4.11.2 NiceGUI dashboard	50
4.12 Evidence-linked operator review	51
4.13 Scalability and operational relevance.....	54
4.14 Limitations.....	56
Chapter 5: Conclusion and Recommendations	58
5.1 Summary of the main project	58
5.2 Unique contribution of the project	59
5.3 Recommendations for future work	60
5.4 Conclusion	62
References.....	63
Appendices	66
Appendix A: Selected core framework excerpts	66
A.1 PatchCore anomaly-detection experiment.....	66
A.2 Pixel-level localisation metrics	67
A.3 Constrained semantic interpretation	68
A.4 Risk-aware human-review gating	69
A.5 PostgreSQL audit-record persistence.....	70
Appendix B: Supplementary operational evidence	72
B.1 PostgreSQL service running through Docker desktop	72
B.2 Selected inspection record and QA disposition	73
B.3 Evidence-linked operator review.....	74
Appendix C: Experimental and computational configuration	75

Appendix D: Reproducibility and repository evidence	76
D.1 Project execution sequence	76
D.2 Repository location and retained project evidence	77

List of Tables

Table 3.1: Methodological design rationale	13
Table 3.2: Dataset categories used in the study	15
Table 3.3: Experimental variables and controls.....	16
Table 3.4: Hardware and software environment.....	18
Table 3.5: Mathematical modelling rationale for PatchCore and PaDiM	21
Table 4.1: Image-level architecture comparison across the selected MVTec AD categories.....	34
Table 4.2: Integrated image-level detection and review results.....	38
Table 4.3: Default localisation performance	40
Table 4.4: Localisation performance after threshold optimisation	42
Table 4.5: Effect of the capsule semantic fallback on human-review workload.....	44
Table 4.6: PostgreSQL inspection-record and review summary	47
Appendix Table C.1: Supplementary implementation parameters for PatchCore and PaDiM.	75

List of Figures

Figure 1.1: Risk-aware inspection pipeline.....	5
Figure 4.1: Image-level AUROC and F1-score for the evaluated PatchCore and PaDiM architectures across bottle, capsule and hazelnut.....	37
Figure 4.2: Docker Compose service status confirming the healthy PostgreSQL audit database.	48
Figure 4.3: Automatically generated OpenAPI documentation for the risk-aware inspection backend.....	49
Figure 4.4: NiceGUI dashboard showing the PostgreSQL inspection-record summary by category and anomaly detection model.....	51
Figure 4.5: Selected capsule inspection record showing the risk-governed QA disposition and latest operator review.	51
Figure 4.6: Evidence-linked operator review panel showing annotated, heatmap, mask and anomaly-map evidence for a capsule inspection record.....	54
Appendix Figure B.1: Docker Desktop showing the containerised PostgreSQL service used by the risk-aware inspection prototype.	72
Appendix Figure B.2: Bottle inspection record showing the automated assessment, review reasons, QA disposition and latest operator review.	73
Appendix Figure B.3: Evidence-linked bottle review panel showing the annotated image, heatmap and predicted mask for the selected inspection case.	74

Acknowledgements

I would like to express my sincere gratitude to my dissertation supervisor, Dr. Dena Bazazian, for her guidance, helpful suggestions and support throughout this research project. Her advice and discussions were valuable in refining the direction of the study, strengthening the experimental methodology and improving the presentation of the completed work. I also acknowledge the University of Plymouth for providing the academic environment and resources that supported the development of this dissertation, as well as MVTec for making the MVTec Anomaly Detection dataset publicly available for research and evaluation.

I am especially grateful to my family for their continued encouragement and support throughout my education. In particular, I would like to thank my parents for their patience, sacrifices and constant belief in my educational development. Their support has been an important source of motivation throughout my studies and during the completion of this dissertation.

Chapter 1: Introduction

1.1 Introduction

Industrial visual inspection is an important component of industrial quality assurance since it aids in the detection of surface flaws, structural abnormalities, and product variations before things go forward in production or distribution. Conventional inspection methods may rely on human judgement or rule-based machine vision technologies. Although these techniques are still valuable in controlled situations, they can be influenced by operator variability, changing lighting, product misalignment, and the complexity of creating rules for every potential problem.

Deep learning techniques have enhanced the ability of inspection systems to learn complex visual data. However, supervised techniques often require representative, labelled instances of each defect class. In industrial contexts, defective samples are often rare, imbalanced, or missing during system development. One-class anomaly detection addresses this restriction by learning the appearance of normal products and detecting test samples that differ from the learned representation. The MVTec Anomaly Detection dataset facilitates this research framework by offering defect-free training images, anomalous test images, and pixel-level defect classifications across numerous industrial categories (Bergmann et al., 2019).

This study looks at an end-to-end risk-aware visual inspection framework using the bottle, capsule, and hazelnut categories from MVTec AD. PatchCore and PaDiM are used as the primary anomaly detection architectures. Both methods employ feature embeddings derived from pre-trained convolutional neural networks, but they represent normality in different ways. PatchCore maintains realistic normal patch embeddings in a memory bank

and uses nearest-neighbour distance during inference (Roth et al., 2022). Using position-specific multivariate Gaussian distributions, PaDiM models normal patch embeddings and uses Mahalanobis distance to determine deviations from normal appearance (Defard et al., 2021).

The study goes beyond classifying images as normal or abnormal. It includes anomaly localisation, constrained semantic interpretation, confidence fusion, deterministic risk classification, human-review gating, and auditable quality resolution. The resulting prototype links the detection pipeline to PostgreSQL storage, a FastAPI backend, and a NiceGUI operator dashboard.

1.2 Motivation of study

The principal motivation for the study is the gap between detecting an anomaly and making a defensible quality-control decision. A high anomaly score shows that an image or region deviates from the learned normal feature distribution; nevertheless, it does not determine the defect type, operational severity, or appropriate manufacturing response. An apparent deviation may represent a genuine defect, an acceptable product variation, an imaging artefact, or a condition that cannot be interpreted reliably without human judgement.

Much of the industrial anomaly detection literature evaluates performance using image-level metrics like AUROC and F1-score, as well as localisation metrics derived from anomaly maps. These metrics are necessary for model comparison, but they do not demonstrate whether the output can be translated into a traceable decision to pass, review, hold, release, or reject a product. They also provide limited information regarding the workload incurred when unclear cases are escalated to an operator.

Another factor is the need to examine different representations of normality rather than assuming that one architecture is generally applicable. PatchCore provides a memory-based model that can retain a variety of normal appearance instances, whereas PaDiM provides a compact statistical representation that considers feature correlations and spatial location. Their comparative evaluation allows the selected architecture to be justified via experimental evidence rather than architectural preferences.

The study is also motivated by the need for accountable AI-assisted inspection. The NIST AI Risk Management Framework focuses on incorporating trustworthiness and risk-management considerations throughout the design, usage, and assessment of AI systems (NIST, 2023). In this dissertation, automated suggestions are supported by visual evidence, confidence components, risk reasoning, and long-term audit records. Cases with weak, unknown, or high-risk evidence can be excluded from automatic resolution and routed to an operator.

1.3 Research problem

Existing anomaly detection methods perform well on controlled industrial benchmarks, but a model-level result does not represent a comprehensive inspection system. Three linked issues remain.

First, strong image-level performance does not imply precise spatial localisation. Normalisation, thresholding, and post-processing are all required for anomaly maps, and the choices made can have a significant impact on the resulting defect mask.

Second, anomaly detection recognises deviations from normal appearance but may not always offer a meaningful semantic description or risk classification. Treating anomaly magnitude as a direct indication of severity may result in unjustified quality decisions.

Third, many experimental pipelines do not preserve the complete pathway from model output to final disposition. Without structured storage of model configuration, localisation evidence, semantic interpretation, risk rules, review reasons, and operator decisions, it may be difficult to determine why an item was released, escalated, or rejected.

The research problem addressed by this dissertation is how to integrate one-class visual anomaly detection with localisation, controlled interpretation, risk-based reasoning, human oversight, and auditability into a single prototype inspection framework.

1.4 Research aim

The aim of this study is to develop and evaluate a risk-aware industrial visual inspection framework that incorporates one-class anomaly detection, defect localisation, constrained semantic interpretation, confidence fusion, deterministic risk assessment, human-review gating, and auditable quality-disposition support.

1.5 Research objectives

The objectives of the study are:

1. To implement and compare PatchCore and PaDiM using alternative convolutional neural network backbones for image-level anomaly detection across the selected MVTec AD categories.
2. To evaluate the localisation performance of the anomaly maps using pixel-level overlap measures and assess the effect of category- and model-specific threshold optimisation.
3. To integrate anomaly and localisation evidence with constrained semantic interpretation, confidence fusion, and deterministic risk-classification rules.

4. To develop a human-review and quality-disposition workflow that is linked to evidence and supported by PostgreSQL audit storage, a FastAPI backend, and a NiceGUI operator interface.
5. To evaluate the framework using both technical measures and operational indicators, including detection performance, localisation quality, human-review rates, and traceability of final decisions.

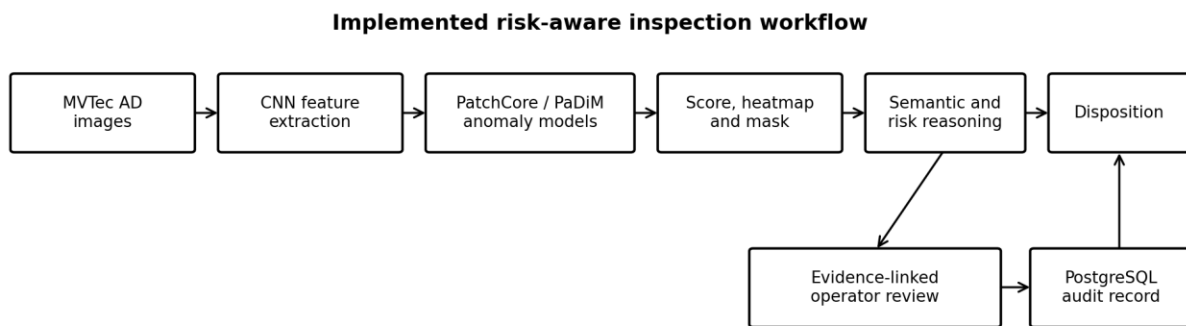


Figure 1.1: Risk-aware inspection pipeline.

Chapter 2: Literature Review

2.1 Introduction

Industrial visual inspection has moved from rule-based image processing towards deep learning because handcrafted systems often require extensive redesign when product appearance, illumination or defect form changes. This shift has improved the ability of inspection systems to learn complex visual patterns, but supervised approaches remain constrained by the scarcity and cost of labelled defect data. In many manufacturing settings, normal products are abundant while defective samples are rare, diverse, or previously unseen. As a result, one-class and unsupervised anomaly detection have become crucial alternatives, as they model normal appearance and recognise abnormalities during inference (Czimmermann et al., 2020; Tao et al., 2022).

This chapter critically assesses the dissertation's most significant research: feature-embedding anomaly detection, localisation, semantic and risk interpretation, human review, and operational traceability. Its goal is not to list every anomaly detection method, but to map the theoretical foundations of the proposed risk-aware inspection framework and identify the gap between strong benchmark performance and auditable quality-control decisions.

2.2 Industrial visual inspection and one-class anomaly detection

Conventional machine-vision systems incorporate explicit characteristics such as colour, texture, edges, and shape, followed by predefined thresholds or classifiers. Their transparency and minimal computational demand are still helpful when the products and acquisition conditions are reasonably stable. However, the rules are often application-specific and sensitive to lighting variability, misalignment, and acceptable variations in

normal appearance (Czimmermann et al., 2020). Supervised convolutional networks decrease the need for manual feature design, yet they imply that the available training set truly captures the defect classes that would occur in operation. This is a serious vulnerability when failures occur infrequently or cannot be expected altogether.

One-class anomaly detection provides a better theoretical foundation for these conditions. It learns a description of normality and considers inadequately substantiated observations to be abnormal. MVTec AD has established a widely used benchmark for this setting by providing defect-free training images, anomalous test images, and pixel-level defect masks across industrial object and texture categories (Bergmann et al., 2019). The dataset allows for independent evaluations of image-level detection and pixel-level localisation, although benchmark performance should not be used to determine deployment readiness because real-world production conditions may include camera drift, changing illumination, and broader normal variation.

The current literature categorises one-class methods as reconstruction, boundary, distribution, and feature-embedding-based approaches. Reconstruction methods derive abnormalities from the failure to replicate normal images, although expressive decoders may also reconstruct defects. In contrast, feature-embedding techniques compare representations derived by pre-trained convolutional networks. They have gained popularity because local patch features maintain minor defect evidence that may be diluted in a global image representation, while avoiding the need to train a large network from the limited industrial dataset (Tao et al., 2022; Liu et al., 2024).

2.3 Theoretical framework: PatchCore and PaDiM

The dissertation compares PatchCore and PaDiM because they represent two different theories of normality within the same feature-embedding framework. Both employ pre-trained CNN backbones to generate an image-level anomaly score and a spatial anomaly map, but they differ in how normal patch embeddings are represented.

PatchCore uses an instance-based interpretation. It stores typical normal patch embeddings in a coreset memory bank and scores each test patch based on its nearest neighbour distance from that bank (Roth et al., 2022). Its key advantage is that it can retain many valid modes of normal appearance without constraining them into a single parametric distribution. Coreset selection reduces memory and search costs, although extreme reduction might remove uncommon but acceptable normal patterns. PatchCore may also be sensitive to isolated responses, and its localisation quality is determined by the spatial resolution of the anomaly map and the selected threshold.

PaDiM uses a distributional and position-specific interpretation. At each feature-map location, it computes the mean and covariance of normal embeddings and scores a test feature against the expected local distribution using the Mahalanobis distance (Defard et al., 2021). This results in a compact model that contains correlations between feature dimensions and can be useful when objects are consistently aligned. Its drawbacks stem from the same assumptions: a single Gaussian may not represent multimodal normal variation; covariance estimation becomes difficult in large dimensions, and benign positional changes may be viewed as abnormal.

Neither architecture appears to be superior in every scenario. PatchCore provides flexible coverage of normal variation but requires memory-searching; PaDiM compresses

normality statistically but is more dependent on distributional and spatial assumptions. As a result, the research suggests that empirical comparison is preferred over model selection based only on reputation. This is consistent with the dissertation's architectural experiments with different backbones and product categories. The comparison also acknowledges that image-level classification and pixel-level localisation are distinct capabilities: a model can accurately classify anomalous images while producing diffuse or poorly thresholded masks.

2.4 From anomaly evidence to risk-aware inspection

Anomaly detection does not qualify as a comprehensive quality decision. A continuous anomaly map must be scaled, normalised, and thresholded before it can be used as a binary localisation mask. Threshold selection can change the balance between missing defects and false regions, implying that localisation performance is influenced by post-processing as well as the detector. The use of IoU and Dice therefore gives evidence of spatial agreement; nevertheless, low scores for small or thin defects may represent boundary sensitivity rather than complete localisation failure.

Another constraint is semantics. PatchCore and PaDiM identify deviations from learnt normality, not the physical identity or consequences of the deviation. A high anomaly score might indicate a scratch, contamination, deformation, misalignment, or acceptable variance. Local heatmaps are frequently used to represent structural abnormalities, whereas logical anomalies may entail an incorrect quantity or arrangement of otherwise normal components (Bergmann et al., 2022). This limitation motivates a separate semantic layer with a constrained defect taxonomy and an explicit unknown state, rather than classifying every anomaly into an unsupported class.

Risk is separate from anomaly magnitude. A visually apparent but superficial mark may have minimal functional significance, but a small defect in a critical location may necessitate escalation. To avoid considering the detector score as a direct measure of severity, the proposed framework maps supported semantic labels to deterministic risk and quality-disposition criteria. Cases with low confidence, unclear status, or high risk are routed for human review.

Human oversight is useful only when the operator has enough evidence, and the action is recorded. Human-AI interaction research focuses on communicating system constraints, providing explanations for suggestions, and facilitating correction (Amershi et al., 2019). In the proposed framework, the review panel connects the original image, heatmap, predicted mask, confidence components, risk analysis, and automated recommendation. The operator decision is recorded independently from the original model output, preserving disagreement for further analysis.

Traceability is equally vital. NIST's AI Risk Management Framework views governance, documentation, and monitoring as continual responsibilities rather than enhancements made after model development (NIST, 2023). Similarly, Sculley et al. (2015) demonstrate that operational machine-learning dependability is dependent on data, configuration, and surrounding software, as well as the model code. These perspectives support the project's PostgreSQL audit database, API, and dashboard, which integrate model outputs, rules, reviews, and final dispositions into a single traceable workflow.

2.5 Research gap and alignment with the proposed study

The reviewed literature demonstrates strong progress in normal-only industrial anomaly detection, particularly on controlled benchmarks. However, three connected gaps remain.

First, evaluation is mostly model-based. AUROC, F1-score, and localisation metrics are useful, but they do not indicate how many records require manual review, why instances are escalated, or whether a prediction can support a defensible quality action. Second, detection, semantic interpretation, and risk tend to be treated separately. A detector can detect visual deviations without determining the defect's meaning or operational consequences. Third, visual explanations are often presented without a permanent record of the configuration, threshold, risk rule, and human decision that resulted in the final outcome.

The dissertation bridges these gaps by integrating comparative PatchCore and PaDiM detection with localisation post-processing, constrained semantic interpretation, confidence fusion, deterministic risk classification, human-review gating, and persistent audit storage. Its contribution is thus not a novel anomaly-detection architecture. It is the design and evaluation of a risk-aware inspection pipeline that assesses both technical performance and operational decision behaviour. The proposed study analyses image-level detection and localisation, measures review workload, retains evidence for operator decisions, and shows the framework using a PostgreSQL-backed FastAPI and NiceGUI interface.

This positioning also defines the limits of the work. The system is a prototype decision-support framework rather than a validated production-line controller. Real deployment would require independent threshold calibration, broader data, usability testing, drift monitoring, latency evaluation and safety validation before connection to physical reject or sorting equipment.

Chapter 3: Methodology

3.1 Introduction

This chapter describes the approach employed to design, implement, and evaluate the risk-aware visual inspection prototype. The study integrated controlled anomaly-detection experiments with development of an operational decision-support framework that includes localisation, constrained semantic interpretation, confidence-based review gating, deterministic risk classification, and auditable quality disposition.

PatchCore and PaDiM were evaluated as the principal anomaly-detection methods using selected categories from the MVTec Anomaly Detection dataset. Their results were then integrated into the broader inspection framework, enabling the evaluation of model-level anomaly evidence alongside localisation quality, semantic interpretation, risk, human review requirements, and traceability. The methodology therefore addressed both the technical performance of the detectors and the conversion of their outputs into reviewable quality-control recommendations.

3.2 Methodological rationale and research design

The research design combined controlled experimental comparison with the prototype development. The experimental component compared anomaly-detection architectures and feature-extraction configurations under consistent dataset conditions, while the prototype component assessed how detector outputs could be incorporated into localisation, interpretation, risk assessment, human review and audit storage.

This design was appropriate because the research problem extended beyond detector accuracy. A modular architecture allowed the anomaly-detection component to be

evaluated independently while retaining a consistent downstream decision-support process. Table 3.1 summarises the principal methodological elements, their rationale and the evidence generated during evaluation.

Table 3.1: Methodological design rationale

Methodological element	Rationale in the project	Resulting evidence
Controlled anomaly-detection experiments	Allowed PatchCore, PaDiM and backbone configurations to be compared under the same dataset organisation.	Image-level AUROC, accuracy, precision, recall and F1-score.
Localisation evaluation	Assessed whether the anomaly maps could provide spatial evidence rather than only image-level classification.	IoU, Dice and threshold-sweep results.
Prototype decision-support layer	Tested how anomaly evidence could be translated into review, risk, and quality disposition.	Review rates, risk classes, QA decisions, and operator-review records.
Audit-oriented software implementation	Preserved the model output, visual evidence, risk reasoning, and human decision for traceability.	PostgreSQL records, JSON payloads, FastAPI endpoints and dashboard evidence.

The prototype was confined to decision support, and did not control a conveyor, reject mechanism or robotic system. Cases with low confidence and high risk remained under

human assessment, preserving a clear boundary between automated evidence creation and ultimate operational judgement (Amershi et al., 2019).

3.3 Data collection rationale and data organisation

3.3.1 Dataset selection

The experimental data was extracted from the MVTec Anomaly Detection dataset. MVTec AD features high-resolution industrial imagery classified into object and texture types, containing defect-free training images, normal and anomalous test images, as well as pixel-level ground-truth masks for the anomalous test cases (Bergmann et al., 2019). It was selected because it supports the normal-only anomaly detection setting used in the dissertation while also allowing image-level and pixel-level evaluation.

The study did not collect proprietary manufacturing images. This decision improved reproducibility, avoided confidentiality restrictions, and allowed the results to be compared with established anomaly-detection research. It also fitted the methodological aim of evaluating a prototype inspection framework before any production-site data collection or industrial deployment.

Three distinct object categories were implemented: bottle, capsule, and hazelnut. These categories were selected primarily for their distinct shapes, textures, structural designs, and defect characteristics. Visual representations of bottles display noticeable faults, including broken regions and contamination. Capsule defects involve squeezes, scratches, cracks, and punctures. Images of hazelnuts exhibit natural variations, featuring surface defects, cuts, cracks, and contamination. Therefore, the selection provided a controlled yet varied evaluation of the identical pipeline across distinct visual conditions.

Table 3.2: Dataset categories used in the study

Category	Reason for inclusion	Methodological value
Bottle	Contains clearly visible object defects and contamination patterns.	Useful for testing whether detectors separate strong structural anomalies from normal object appearance.
Capsule	Contains small, thin and localised defects such as squeeze, scratch, crack and poke examples.	Tests sensitivity to subtle local evidence and localisation threshold choice.
Hazelnut	Contains natural object texture variation together with cuts, cracks and contamination.	Tests robustness where normal appearance is more variable, and feature distributions are less uniform.

3.3.2 Data partitioning and use of ground truth

The official folder arrangement of MVTec AD was maintained. The train/good directory images of each category were used to evaluate the normality representation of PatchCore and PaDiM. These images contained no labelled defects. Normal and anomalous samples contained in the test directories were used for evaluation. Ground-truth masks were solely employed for the purpose of localisation evaluation and were not provided to the anomaly detection models during either fitting or inference.

Adopting the official split maintained the one-class assumption and avoided contamination between training and evaluation. Individual output folders were generated for each category, detector, and backbone setup, ensuring that experimental artefacts were not overwritten. The same category splits were used in every architectural

comparison; only the detector architecture or clearly specified parameter was altered between test runs. This lessened uncontrolled variation and ensured that the comparison was linked to the tested configuration rather than changes in data selection.

3.4 Experimental design

The experimental design was organised into three levels of evaluation. The first assessed anomaly detection at image level, the second examined pixel-level localisation, and the third evaluated the operational behaviour of the integrated framework, including semantic uncertainty, human-review workload, risk classification and traceability. This structure allowed detector performance to be assessed separately from the downstream decision-support stages.

PatchCore was evaluated using ResNet18, ResNet50 and WideResNet50-2 backbones. PaDiM was evaluated using ResNet18 with its default retained-feature configuration and ResNet50 with 100 retained features. The same MVTec AD category splits were used for bottle, capsule and hazelnut across the architectural comparisons. Batch size, output organisation and evaluation procedures were kept consistent within the relevant experiments, subject to the limitations of the CPU-based computational environment.

Table 3.3: Experimental variables and controls

Aspect	Varied in the experiments	Controlled during comparison
Detector architecture	PatchCore and PaDiM.	Same MVTec AD category splits and evaluation procedure.
Feature backbone	ResNet18, ResNet50 and WideResNet50-2 supported.	Same output structure and image-level metrics.

Aspect	Varied in the experiments	Controlled during comparison
PaDiM retained features	ResNet18 default and ResNet50 with n_features = 100.	Consistent retained-feature value across categories for ResNet50.
Localisation threshold	Candidate thresholds were tested during sweeps.	Same saved anomaly maps and ground-truth masks for each category/model pair.
Operational decision logic	Semantic fallback and confidence/risk rules were evaluated.	Same database schema, dashboard, and QA-disposition logic.

The controlled comparison was designed to provide an evidence-based basis for detector and backbone selection rather than assuming the suitability of a particular architecture from the literature. The resulting architecture-selection evidence is presented and interpreted in Chapter 4.

3.5 Software frameworks and computational environment

The experiments and prototype were conducted locally in Python on a Windows 11 Pro workstation. The project dependencies were managed in a dedicated virtual environment. No CUDA-enabled GPU was available; therefore, feature extraction, model fitting, and inference were performed on the CPU. This computational constraint is crucial for the interpretation of execution time and scalability as the provided experiments are not representative of GPU-accelerated or production-line performance.

Table 3.4: Hardware and software environment

Component	Specification used in the project
Processor	Intel Core i5-8250U CPU at 1.60 GHz; 4 physical cores and 8 logical processors.
Installed memory	7.63 GB reported physical memory, corresponding to a nominal 8 GB RAM configuration.
Graphics processor	Intel UHD Graphics 620; driver version 23.20.16.5018.
CUDA availability	Not available to PyTorch; all experiments used CPU execution.
Operating system	Microsoft Windows 11 Pro, 64-bit, version 10.0.22000, build 22000.
Python environment	Python 3.13.5 is installed in a dedicated .venv_anomalib virtual environment.
PyTorch	2.8.0+cpu.
Anomalib	2.5.0.
OpenCV	4.13.0.
Database and application layer	PostgreSQL 16 through Docker Compose, FastAPI backend and NiceGUI dashboard.

Anomalib was selected because of its robust implementations and supportive utilities for industrial anomaly detection and localisation, such as PatchCore and PaDiM (Akçay et al., 2022). The underlying deep-learning execution framework was provided by PyTorch, while OpenCV supported the post-processing of anomaly maps and the generation of

visual evidence. PostgreSQL, FastAPI, and NiceGUI formed the operational application layer for persistent audit storage, backend accessibility, and operator review.

3.6 Detection pipeline and mathematical modelling

3.6.1 Shared feature-extraction approach

PatchCore and PaDiM were implemented as normal-only anomaly-detection methods using feature representations derived from pre-trained residual convolutional neural networks. Intermediate feature maps were obtained from normal training images and used to construct the corresponding representations of normal product appearance. Residual networks were chosen as feature extractors due to their skip connections, which facilitate the optimisation of deep visual representations (He et al., 2016).

The feature extractors were not trained on MVTec AD. Instead, normal images were propagated through selected intermediate layers, and the resulting local feature embeddings were used by PatchCore or PaDiM to model normality. This approach aligned with the one-class experimental setting, in which only defect-free images were available during model fitting.

3.6.2 PaDiM distribution modelling

By calculating a multivariate Gaussian distribution for the normal feature embeddings at each spatial position, PaDiM statistically represented normality (Defard et al., 2021).

Let x_p denote a patch embedding at spatial position p , μ_p the estimated mean normal embedding and Σ_p the corresponding covariance matrix. The normal feature distribution was represented as:

$$x_p \sim \mathcal{N}(\mu_p, \Sigma_p)$$

During inference, the deviation of a test embedding from the learned normal distribution was measured using the Mahalanobis distance:

$$d_p = \sqrt{(x_p - \mu_p)^T \Sigma_p^{-1} (x_p - \mu_p)}$$

A larger value of d_p indicates greater deviation from the normal feature distribution expected at that spatial position. The collection of spatial distances was used to construct the anomaly map, which was subsequently resized to the input-image resolution for localisation.

PaDiM was evaluated using ResNet18 with its default feature configuration and ResNet50 with 100 retained features. The ResNet50 retained-feature value remained constant across bottle, capsule, and hazelnut, ensuring that category comparisons were performed under the same feature-dimensionality setting.

3.6.3 PatchCore memory-bank modelling

A memory bank of local patch embeddings from the normal training images was used by PatchCore to represent normality (Roth et al., 2022).

Let $\mathcal{M}$ denote the normal memory bank and q a patch embedding extracted from a test image. The basic anomaly score for q was determined from its Euclidean distance to the nearest retained normal embedding:

$$s(q) = \min_{m \in \mathcal{M}} \|q - m\|_2$$

A small nearest-neighbour distance indicates a similarity to the learned normal feature space, whereas a larger distance indicates the test patch is inadequately represented by the retained normal examples. Patch-level scores were spatially arranged to form the

anomaly map used for subsequent localisation, whilst the strongest responses contributed to the image-level anomaly assessment.

PatchCore was evaluated using ResNet18, ResNet50, and WideResNet50-2 architectures. Unless deliberately modified for a specific comparison, the coreset sampling ratio remained at 0.1 and the number of nearest neighbours was set at nine.

Table 3.5: Mathematical modelling rationale for PatchCore and PaDiM

Detector	Representation of normality	Anomaly scoring principle	Reason for inclusion
PatchCore	Representative normal patch embeddings retained in a coreset memory bank.	Nearest-neighbour distance between a test patch and the closest retained normal embedding	Evaluates a memory-based representation capable of retaining diverse examples of normal local appearance.
PaDiM	Position-specific multivariate Gaussian distributions estimated from normal patch embeddings.	Mahalanobis distance between a test embedding and the expected normal distribution at that spatial position.	Evaluates a compact statistical representation incorporating feature correlations and spatial position.

3.7 Anomaly localisation and post-processing

PatchCore and PaDiM generated continuous anomaly maps representing the relative strength of abnormality across each image rather than binary defect masks. A consistent post-processing procedure was thus applied to convert these maps into localised regions suitable for evaluation and subsequent interpretation.

Each anomaly map was first normalised to a common range. A configurable threshold, τ , was then applied to produce a binary mask. For anomaly-map value $A(x,y)$, the corresponding mask $M(x,y)$ was defined as:

$$M(x,y) = \begin{cases} 1, & A(x,y) \geq \tau \\ 0, & A(x,y) < \tau \end{cases}$$

Morphological opening and closing were applied where required. Opening reduced isolated foreground noise, while closing filled small gaps within detected regions. Connected-component analysis then grouped foreground pixels into coherent anomalous regions. Components below the configured minimum-area threshold were removed. For each retained component, the pipeline extracted its area, bounding box, relative mask coverage, spatial position and local anomaly evidence.

Threshold-sweep experiments were included because localisation performance depends both on the quality of the continuous anomaly map and on the threshold used to convert it into a binary mask. Candidate thresholds were applied to saved anomaly maps, and the resulting masks were compared with the MVTec AD ground-truth masks using Intersection over Union and Dice scores. This procedure allowed the effect of the binarisation threshold to be examined separately from the detector's underlying anomaly representation.

3.8 Constrained semantic interpretation

The semantic interpretation stage converted localised visual evidence into a restricted set of manufacturing-relevant defect descriptions. It did not use open-ended image captioning or unrestricted text generation. Instead, the implementation used a constrained taxonomy containing the labels scratch, crack, dent, contamination, deformation, surface defect and unknown.

Interpretation was based on evidence extracted from the selected connected component and its surrounding image region. This evidence included component area, aspect ratio, spatial location, mean intensity, saturation contrast, edge density, anomaly-map response and relative mask coverage. The assigned label and its supporting evidence were stored together so that the interpretation could subsequently be reviewed.

When the available evidence was insufficient or inconsistent, the system returned unknown. This was treated as an explicit uncertainty state rather than a classification failure. Unknown or weakly supported interpretations were therefore directed to human review by the governance logic.

A capsule-specific fallback mechanism was also implemented for cases in which the evidence supported a broader defect family but did not justify a more specific label. The fallback remained restricted to the predefined taxonomy and did not modify the anomaly detector, anomaly score or localisation mask. Its operational effect was evaluated subsequently through the human-review rate.

3.9 Risk determination and confidence fusion

3.9.1 Deterministic risk classification

Risk classification was performed using deterministic rules instead of a generative or learned decision model. This ensured that identical inputs produced consistent results and that every assigned risk class could be traced to an explicit policy. The risk logic considered the constrained semantic label, anomaly evidence, localisation evidence and the configured risk policy. The available classes were Low, Medium, High, Critical and Review Required.

The framework was guided by the principles of Failure Mode and Effects Analysis, wherein potential failures are systematically examined to support in prioritising and treatment decisions (International Electrotechnical Commission, 2018). The prototype was not designed to replace a manufacturer-specific FMEA executed by qualified domain experts. Instead, it demonstrated how a clear risk matrix could be systematically applied after visual anomaly detection.

Risk mappings were applied solely where the available evidence supported a defined rule. Missing, uncertain or unsupported mappings were not estimated. These cases were assigned Review Required so that the system did not convert incomplete evidence into an unsupported automated quality decision.

3.9.2 Confidence fusion

Three evidence components were represented during confidence fusion: anomaly confidence, localisation confidence and semantic confidence. Anomaly confidence, C_a , represented the strength of the detector response relative to its decision boundary. Localisation confidence, C_l , represented the consistency and strength of the selected

anomalous region. Semantic confidence, C_s , represented the evidence supporting the assigned constrained label.

Two transparent fusion strategies were implemented. The conservative strategy used the minimum component confidence:

$$C_{\min} = \min(C_a, C_l, C_s)$$

This strategy prevented a strong confidence value from one stage from hiding weak evidence in another. A low value in any component therefore reduced the overall confidence and increased the likelihood of review.

The second strategy used a weighted average:

$$C_w = \frac{w_a C_a + w_l C_l + w_s C_s}{w_a + w_l + w_s}$$

where w_a , w_l and w_s were the configured weights assigned to anomaly, localisation and semantic confidence, respectively. This provided a less conservative alternative when the relative reliability of the components justified different contributions. The fused value was not interpreted as a calibrated probability of defect occurrence; it was used as a transparent operational-review gate.

An inspection record was routed to human review when one or more configured conditions were met: fused confidence below the operational threshold, an unknown semantic label, High or Critical risk, incomplete risk mapping, or otherwise ambiguous evidence. The reason for escalation was stored with the inspection record so that the operator could identify why the case required review.

3.10 Quality disposition and audit storage

The quality-disposition layer translated the available evidence into a practical recommendation. Possible actions included pass, pass with monitoring, reinspect, hold, quarantine, review required, release after review, reject after review and engineering review. The disposition combined the automated inspection record with the most recent operator review. An uncertain item could initially receive REVIEW_REQUIRED; if an operator subsequently confirmed a false positive, the status could change to RELEASE_AFTER_REVIEW. Conversely, a confirmed defect could remain held or be rejected according to the recorded decision and risk class.

PostgreSQL was employed to store inspection records and operator reviews. The main inspection table maintained structured fields including image identifier, category, model name, semantic label, risk class, confidence metrics, review requirement, and timestamp. Additional pipeline evidence was retained in a JSONB result payload to maintain component-specific details and configuration summaries without necessitating a distinct relational column for each variable.

Operator reviews were stored in a related table using a foreign-key reference to the inspection record. Each review retained the operator's decision, review note, reviewer identifier, and timestamp. This design maintained the original automated result while permitting one or more human review instances linked to it. The database was populated with 650 inspection records obtained from the PatchCore and PaDiM experiments involving bottles, capsules, and hazelnuts.

3.11 Operational application layer

The operational application layer was implemented to demonstrate how the audit database could support reviewable inspection decisions. PostgreSQL 16 was deployed as a Docker Compose service. The container exposed PostgreSQL to the host through port 55432 while retaining PostgreSQL's internal port 5432. A health check was used to confirm that the service was ready before database-dependent operations were performed.

FastAPI provided the backend interface. The implemented endpoints included GET /health, GET /records, GET /records/{record_id}, GET /records/{record_id}/qa-decision, POST /reviews, and GET /summary. These routes supported health checking, record listing, individual-record retrieval, review submission, QA-disposition retrieval, and database summaries. The automatically generated OpenAPI documentation was used during implementation to test backend functions.

NiceGUI provided the browser-based operator dashboard. The dashboard displayed database summaries, filterable inspection records, selected-record details, QA disposition, the latest operator review, and linked visual artefacts. The evidence panel connected a selected image identifier to available annotated images, heatmap overlays, predicted masks, and anomaly maps. This established a traceable link between model evidence, operator judgement, and the final quality disposition.

3.12 Evaluation methodology

3.12.1 Image-level detection metrics

The evaluation of image-level anomaly detection was conducted using AUROC, accuracy, precision, recall, and F1-score. AUROC examined the efficacy of anomaly

scores in prioritising anomalous images over normal ones across several decision thresholds, thus facilitating the comparison of detectors independent of a singular operational point (Fawcett, 2006). Accuracy measured the overall proportion of correct decisions, but precision and recall were retained because accuracy alone can obscure the balance between false rejection of normal products and missed anomalous products.

Precision was calculated as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall was calculated as:

$$\text{Recall} = \frac{TP}{TP + FN}$$

The F1-score was calculated as:

$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.12.2 Localisation metrics

Pixel-level localisation was assessed using intersection over union (IoU) and the Dice coefficient. For these calculations, true positives represented anomalous pixels present in both the predicted mask and the ground-truth mask. False positives were pixels predicted as anomalous outside the ground truth, while false negatives were ground-truth anomalous pixels that were missed by the prediction.

$$IoU = \frac{TP}{TP + FP + FN}$$

and,

$$Dice = \frac{2TP}{2TP + FP + FN}$$

Mean and median localisation scores were documented due to the variability of results across different defect types and image instances. The inclusion of Dice is due to its ability to emphasise the overlapping area between two masks (Dice, 1945).

3.12.3 Operational evaluation

The operational evaluation measured how the complete pipeline behaved after anomaly detection, localisation, semantics, confidence and risk rules were applied. The human-review rate was calculated as:

$$\text{Review rate} = \frac{N_{\text{review required}}}{N_{\text{inspection records}}}$$

This metric measured the proportion of inspection instances that could not be resolved automatically. The significance lies in the fact that two detectors with comparable image-level scores may yield different workloads when uncertainty, semantic labels, and risk rules are taken into account. The assessment additionally investigated unknown semantic labels, semantic fallback effects, risk-class distribution, quality dispositions, database persistence, operator-review submissions, and evidence traceability.

3.13 Experimental sequence

Experiments were carried out step by step. The statistical detector was originally used to validate end-to-end ingestion and output generation and governance logic. Then, PatchCore and PaDiM were integrated for bottle, capsule and hazelnut. Image-level metrics and review rates were compared, and localisation was evaluated using IoU and

Dice. Finally, threshold sweeps were carried out to determine whether the mask alignment could be improved by post-processing. The semantic fallback behaviour, risk and confidence logic, QA disposition, PostgreSQL persistence, FastAPI routes, and the NiceGUI dashboard were then validated. Final detector selection was supported by a comparison of alternative PatchCore and PaDiM backbones.

This order ensured that the model-level experiments were set up prior to the evaluation of the operational layer. This also meant that the software and database components could be assessed after the detector outputs had been made consistent.

3.14 Reliability, validity and reproducibility

Internal consistency was reinforced by employing identical dataset categories, an official folder structure, and consistent test partitions across various detector configurations. Architecture comparisons changed the selected model or explicitly stated hyperparameter while keeping the evaluation data fixed.

Construct validity was corroborated through the assessment of the prototype across various levels. The AUROC and F1 score evaluated anomaly detection at the image level. Intersection over Union (IoU) and the Dice coefficient were used to evaluate spatial localisation. The human-review rate assessed the impact of uncertainty and governance regulations on operations. Database and dashboard evaluations examined traceability and application integration.

Reproducibility was ensured through configuration-controlled thresholds and policies, distinct output directories, retained CSV and JSONL results, saved visual artefacts, architecture-comparison summaries, version-controlled scripts, documented execution commands, and a Docker Compose database service. The primary constraint on external

validity stemmed from the use of three categories derived from a single benchmark dataset. The results thus endorse the conclusions regarding the executed prototype and selected MVTec AD categories; nonetheless, they should not be generalised to all industrial products or imaging conditions without further validation.

3.15 Ethical and governance considerations

This study did not involve the collection of personal data nor experiments involving human subjects. The operator's identity used in testing was a demonstrational identifier, not personal research data. The primary governance issue was the potential for automated anomaly evidence to support acceptance or rejection of a product. The design thus maintained human oversight, stored reasons for review escalation, and preserved visual evidence with each decision.

The system was not closed to uncertainty. Unknown labels, incomplete risk mappings, and low confidence instances were escalated rather than turned into unsupported automatic decisions. Operator reviews were stored separately from the original model output, preserving the distinction between automated recommendation and human judgement. The prototype should not be used as a safety-critical production controller without further dataset validation, cybersecurity controls, access management, latency testing, domain-expert risk calibration, and industrial-hardware integration testing.

3.16 Chapter summary

This chapter summarised the methodology used to design and evaluate the risk-aware visual inspection prototype. The study used the bottle, capsule, and hazelnut categories of the MVTec Anomaly Detection dataset, which provides reproducible industrial images, defect-free training samples, anomalous test cases, and pixel-level masks for localisation

evaluation. This allowed the project to test anomaly detection on normal-only data without using proprietary manufacturing data, while still providing a controlled basis for comparing image-level detection and defect localisation.

The experimental design compared PatchCore and PaDiM as two different feature-embedding techniques for modelling the normal product appearance. PaDiM adopted position-specific multivariate Gaussian modelling and Mahalanobis distance to model normality. PatchCore used a memory bank of normal patch embeddings and the nearest-neighbour distance to model normality. The models were evaluated using image-level metrics, localisation measures and operational review behaviour across different backbone configurations, hence the preferred detector was selected from evidence in the project rather than assumed from the literature.

The methodology also applied the wider inspection framework around the anomaly detectors. Continuous anomaly maps were converted to binary masks. The masks were analysed using a constrained semantic layer, combined with confidence and risk rules, and routed for operator review when the evidence was uncertain, low confidence, or high risk. PostgreSQL was used to store inspection and review records. FastAPI was used to offer backend access, and NiceGUI was used for the operator's dashboard. The chapter demonstrated how the project developed from anomaly detection to a traceable, risk-aware quality decision-support workflow.

Chapter 4: Results and Discussion

4.1 Introduction

This chapter presents evaluation of the risk-aware visual inspection prototype using the bottle, capsule, and hazelnut categories from the MVTec AD dataset. The analysis differentiates between detector-level performance and integrated-pipeline performance, as strong image classification does not inherently produce effective localisation or a manageable human-review workload.

The chapter first compares the PatchCore and PaDiM backbone configurations using image-level AUROC and F1-score. It then evaluates the selected configurations in the complete prototype using classification, localisation and human-review measures, before examining the audit database, API, dashboard, quality-disposition logic and evidence-linked review workflow.

4.2 Overview of the implemented inspection pipeline

The implemented prototype connected anomaly detection to a traceable decision-support workflow. For each inspection, PatchCore or PaDiM produced an image-level anomaly score and an anomaly map. The downstream stages transformed this evidence into localisation outputs, a constrained defect label, a risk category and a confidence assessment. Low-confidence, semantically uncertain, incomplete-risk or high-risk cases were directed to operator review.

Inspection evidence, model outputs and review outcomes were retained in PostgreSQL and exposed through FastAPI. The NiceGUI dashboard linked each inspection record to its annotated image, heatmap, predicted mask and anomaly map, allowing the operator

to examine the evidence before confirming a quality disposition. The prototype generated software recommendations rather than controlling production equipment directly.

4.3 Anomaly Detection Architecture Comparison

4.3.1 Experimental configuration

The evaluation examined the effect of backbone selection on the image-level performance of PatchCore (Roth et al., 2022) and PaDiM (Defard et al., 2021). PatchCore was evaluated using ResNet18, ResNet50, and WideResNet50-2, whereas PaDiM employed ResNet18 with its standard feature setup and ResNet50 with 100 preserved features. All configurations were examined using similar categories of bottles, capsules, and hazelnuts from MVTec AD (Bergmann et al., 2019). Image-level AUROC measured anomaly-ranking performance across thresholds, while F1-score measured the precision-recall balance at the operating threshold.

Table 4.1: Image-level architecture comparison across the selected MVTec AD categories

Category	Model	Backbone	Retained features	Image AUROC	Image F1-score
Bottle	PatchCore	ResNet18	Not applicable	1.0000	0.9920
Bottle	PatchCore	ResNet50	Not applicable	1.0000	0.9920
Bottle	PatchCore	WideResNet50-2	Not applicable	1.0000	0.9920
Bottle	PaDiM	ResNet18	Default	0.9976	0.9839

Category	Model	Backbone	Retained features	Image AUROC	Image F1-score
Bottle	PaDiM	ResNet50	100	0.9952	0.9841
Capsule	PatchCore	ResNet18	Not applicable	0.9817	0.9767
Capsule	PatchCore	ResNet50	Not applicable	0.9936	0.9818
Capsule	PatchCore	WideResNet50-2	Not applicable	0.9932	0.9770
Capsule	PaDiM	ResNet18	Default	0.8975	0.9463
Capsule	PaDiM	ResNet50	100	0.9110	0.9469
Hazelnut	PatchCore	ResNet18	Not applicable	0.9986	0.9787
Hazelnut	PatchCore	ResNet50	Not applicable	1.0000	0.9928
Hazelnut	PatchCore	WideResNet50-2	Not applicable	1.0000	0.9928
Hazelnut	PaDiM	ResNet18	Default	0.7954	0.8330
Hazelnut	PaDiM	ResNet50	100	0.7761	0.7831

4.3.2 PatchCore performance

PatchCore produced the strongest and most consistent image-level results. Bottle performance was unchanged across the three backbones, whereas capsule and hazelnut benefited from the deeper feature extractors. ResNet50 produced the strongest capsule

combination and matched the best hazelnut result. WideResNet50-2 offered no measurable improvement over ResNet50 and returned a lower capsule F1-score, showing that additional backbone capacity did not automatically improve the classification balance.

4.3.3 PaDiM performance

PaDiM remained competitive on bottle but was more sensitive to product category. ResNet50 modestly improved capsule AUROC without materially changing F1-score, while the hazelnut results declined. Its weaker image-level performance on capsule and hazelnut contrasted with its stronger default localisation results reported later, so PaDiM remained useful as a comparison detector and localisation reference.

4.3.4 Cross-category interpretation

Figure 4.1 presents the architecture comparison across all three categories. The visual comparison shows that the PatchCore configurations remained consistently close to the upper limit of the scoring range, whereas PaDiM performance varied more substantially by category.

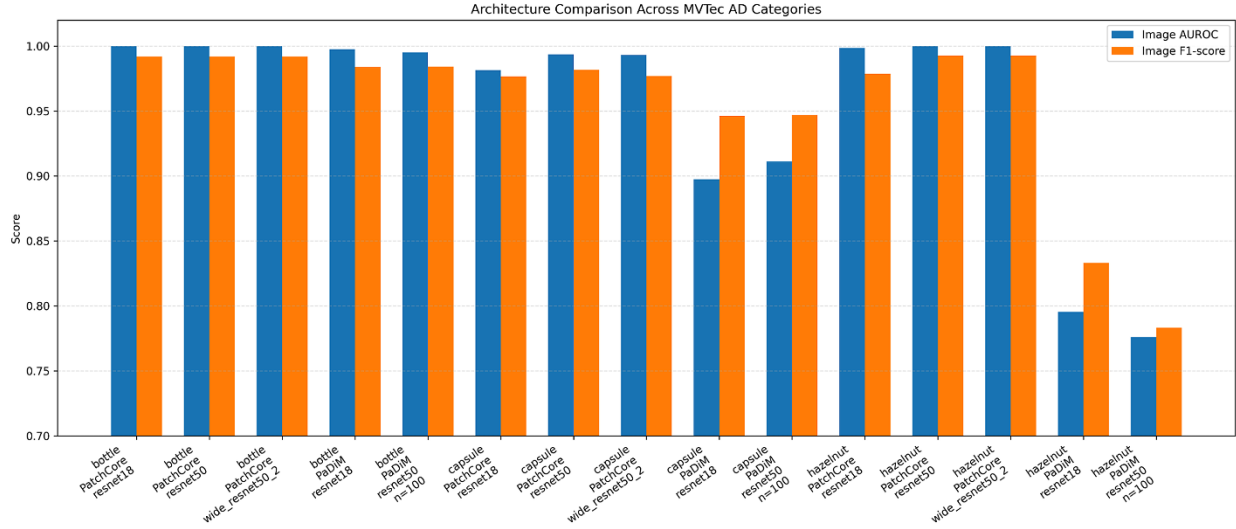


Figure 4.1: Image-level AUROC and F1-score for the evaluated PatchCore and PaDiM architectures across bottle, capsule and hazelnut.

Across the evaluated categories, backbone size alone did not predict performance. PatchCore remained comparatively stable, whereas PaDiM varied more substantially by category. The comparison therefore supports selection on cross-category consistency rather than architectural size alone.

The comparison therefore supported PatchCore as the main image-level detection branch. However, the following results assess localisation quality and human-review behaviour separately because these operational properties cannot be inferred from AUROC and F1-score alone.

4.4 Selection of the preferred detection architecture

PatchCore with ResNet50 was selected as the preferred image-level configuration because it provided the strongest overall balance across bottle, capsule and hazelnut. It produced the best tested combination on capsule and matched the best results on bottle

and hazelnut, with no competing configuration performing better across the complete comparison.

WideResNet50-2 did not provide a sufficient performance advantage to justify its additional capacity, while ResNet18 was weaker on capsule and hazelnut. ResNet50 therefore offered the most appropriate compromise between feature quality, cross-category consistency and computational demand in the available CPU-based environment.

PaDiM was retained as a comparative baseline and localisation reference. The selection of PatchCore with ResNet50 was therefore limited to the preferred image-level detection branch and did not imply that PaDiM was excluded from the wider evaluation.

4.5 Integrated image-level detection results

The architecture comparison reported model-level AUROC and F1-score, while the integrated evaluation examined how detector outputs affected the wider inspection workflow. Table 4.2 therefore includes classification measures together with the proportion of records routed for human review.

4.5.1 Bottle results

Table 4.2 presents the integrated results for the bottle and hazelnut categories.

Table 4.2: Integrated image-level detection and review results

Category	Detector	Accuracy	Precision	Recall	F1-score	Human-review rate
Bottle	Statistical original	0.5904	1.0000	0.4603	0.6304	0.5181

Category	Detector	Accuracy	Precision	Recall	F1-score	Human-review rate
Bottle	Statistical tuned 0.7	0.8795	0.8955	0.9524	0.9231	0.7711
Bottle	PatchCore integrated	0.9880	1.0000	0.9841	0.9920	0.1928
Bottle	PaDiM integrated	0.9518	0.9538	0.9841	0.9688	0.6867
Hazelnut	Statistical original	0.4364	0.8333	0.1429	0.2439	0.2273
Hazelnut	Statistical tuned 0.7	0.5545	0.6567	0.6286	0.6423	0.7636
Hazelnut	PatchCore integrated	0.9909	1.0000	0.9857	0.9928	0.1727
Hazelnut	PaDiM integrated	0.7182	0.6970	0.9857	0.8166	0.9000

Threshold tuning substantially improved the statistical detector's bottle recall and F1-score, but the review rate also increased. PatchCore achieved the strongest integrated bottle result, combining an F1-score of 0.9920 with a review rate of 0.1928. PaDiM matched PatchCore's recall of 0.9841 but produced lower precision and a review rate of 0.6867. The comparison shows that recall alone was insufficient for operational model selection.

4.5.2 Hazelnut results

The same distinction was more pronounced for hazelnut. Although threshold tuning improved the statistical detector, its review rate rose to 0.7636. PatchCore combined an F1-score of 0.9928 with the lowest review rate of 0.1727. PaDiM retained high recall but its lower precision produced an F1-score of 0.8166 and a review rate of 0.9000, indicating a substantially greater false-positive and uncertainty burden.

4.5.3 Cross-category operational interpretation

Across bottle and hazelnut, PatchCore provided the strongest balance between detection performance and review efficiency, with F1-scores above 0.99 and review rates below 0.20. PaDiM's high recall did not translate into comparable operational efficiency because more records required review. These results support the selection of PatchCore by showing that its advantage extended beyond image-level classification to the workload generated by the integrated pipeline.

4.6 Localisation Performance

Pixel-level localisation was evaluated separately because correct image classification does not ensure accurate agreement with the labelled defect region. Table 4.3 shows the intersection over union (IoU) and Dice coefficient scores calculated using the original threshold for mask generation (Dice, 1945).

Table 4.3: Default localisation performance

Category	Model	Mean IoU	Mean Dice	Median IoU	Median Dice
Bottle	PatchCore	0.3472	0.4928	0.3480	0.5164
Bottle	PaDiM	0.4968	0.6442	0.4930	0.6604

Category	Model	Mean IoU	Mean Dice	Median IoU	Median Dice
Hazelnut	PatchCore	0.1456	0.2316	0.1070	0.1933
Hazelnut	PaDiM	0.3084	0.4444	0.3168	0.4811
Capsule	PatchCore	0.1068	0.1671	0.0450	0.0862
Capsule	PaDiM	0.1731	0.2673	0.1365	0.2403

4.6.1 Bottle localisation

Bottle produced the strongest default localisation results. PaDiM achieved a mean IoU of 0.4968 and a mean Dice score of 0.6442, compared with 0.3472 and 0.4928 for PatchCore. The median results followed the same ordering, while PaDiM's similar mean and median values indicated comparatively stable mask alignment.

4.6.2 Hazelnut localisation

PaDiM also produced stronger default localisation on hazelnut, with a mean IoU of 0.3084 and a mean Dice score of 0.4444, compared with 0.1456 and 0.2316 for PatchCore. PatchCore's median IoU was notably below its mean, indicating that several images had weak overlap and that a smaller number of stronger results raised the average.

4.6.3 Capsule localisation

Capsule was the most difficult category for both models. PaDiM achieved a mean IoU of 0.1731 and a mean Dice score of 0.2673, while PatchCore achieved 0.1068 and 0.1671, respectively. The small, narrow and visually subtle defects made precise binary-mask overlap difficult even when the general anomalous region was identified.

4.6.4 Comparison of image-level and pixel-level performance

The default results exposed a clear trade-off: PatchCore was stronger for image-level detection and review efficiency, whereas PaDiM initially produced better spatial alignment. Because each continuous anomaly map had to be converted into a binary mask, the default comparison was also dependent on the selected threshold. Threshold optimisation was therefore required before reaching a final conclusion about localisation quality.

4.7 Localisation optimisation threshold

A category- and model-specific threshold sweep was used to determine whether the continuous anomaly maps contained spatial information that was not captured by the original binary conversion. Table 4.4 reports the best mean IoU and Dice results obtained in this analysis.

Table 4.4: Localisation performance after threshold optimisation

Category	Model	Best threshold	Mean IoU	Mean Dice
Bottle	PatchCore	0.70	0.5929	0.7322
Bottle	PaDiM	0.35	0.5110	0.6612
Hazelnut	PatchCore	0.80	0.4331	0.5854
Hazelnut	PaDiM	0.50	0.3340	0.4693
Capsule	PatchCore	0.85	0.2979	0.4311
Capsule	PaDiM	0.55	0.1830	0.2751

4.7.1 Bottle threshold optimisation

Threshold optimisation raised PatchCore's bottle mean IoU and Dice to 0.5929 and 0.7322, respectively, exceeding PaDiM's tuned results of 0.5110 and 0.6612. This indicates that the original PatchCore threshold, rather than the absence of spatial evidence, limited its default mask alignment.

4.7.2 Hazelnut threshold optimisation

PatchCore showed the largest improvement on hazelnut and achieved a mean IoU of 0.4331 and a mean Dice score of 0.5854. PaDiM improved more modestly and remained below the tuned PatchCore result, showing that a more selective threshold better isolated the strongest PatchCore responses.

4.7.3 Capsule threshold optimisation

Capsule remained the most challenging category, but PatchCore again benefited more from optimisation. Its tuned mean IoU and Dice rose to 0.2979 and 0.4311, compared with 0.1830 and 0.2751 for PaDiM. The result suggests that the PatchCore map identified relevant capsule regions but required a higher threshold to limit mask expansion beyond the small defect areas.

4.7.4 Implications of threshold tuning

After tuning, PatchCore produced the strongest mean localisation result in all three categories, reversing the ordering observed under the default thresholds. This demonstrates that anomaly-map quality and binary-mask quality should not be treated as identical and that a single universal threshold was not suitable for the three product categories.

The tuned results should nevertheless be interpreted cautiously because the thresholds were selected using the available ground-truth masks. They demonstrate the potential benefit of calibration but may be optimistic. A production study should select thresholds on a separate validation set and report final performance on previously unseen test images.

4.8 Semantic interpretation and review workload

The semantic stage assigned localisation evidence to a constrained defect taxonomy and routed unresolved cases for review. For capsule, a category-specific fallback was introduced when the evidence supported a broader permitted defect category. The anomaly detectors were not retrained and their classification outputs were unchanged.

Table 4.5 compares the capsule review rates before and after the fallback. The reported integrated F1-scores are distinct from the model-level architecture scores in Table 4.1 and were unaffected by the semantic rule.

Table 4.5: Effect of the capsule semantic fallback on human-review workload

Detector	Integrated F1-score	Review rate before	Review rate after
PatchCore capsule	0.9815	0.6818	0.5152
PaDiM capsule	0.9432	0.8712	0.8409

The fallback reduced the PatchCore review rate by 16.66 percentage points and the PaDiM rate by 3.03 percentage points without changing integrated F1-score. The larger PatchCore reduction indicates that more of its review cases resulted from semantic uncertainty that the constrained fallback could resolve. PaDiM retained a substantially

higher review rate, suggesting that other uncertainty or localisation conditions continued to trigger review.

The fallback did not generate unrestricted defect descriptions or suppress review when the evidence remained weak. The result therefore demonstrates that review workload can be reduced within the semantic and governance stages without altering detector performance, while retaining operator oversight for unresolved cases.

4.9 Risk-governed quality disposition

The results of the anomaly detection were not regarded as the conclusive quality-control decision. The implemented risk-governed disposition layer combined the anomaly result, localised evidence, semantic interpretation, confidence values, risk classification, and any accessible operator review to arrive at a suitable operational recommendation.

The initial disposition depended on the evidence available at the time of inspection. Records with sufficiently strong and consistent evidence could receive outcomes such as *PASS* or *PASS_WITH_MONITORING*. Cases involving low confidence, an unknown semantic label, an incomplete risk mapping or a high-risk classification were assigned *REVIEW_REQUIRED*, *HOLD*, *QUARANTINE* or *ENGINEERING_REVIEW*, depending on the configured conditions.

This design prevented the anomaly detector from autonomously executing an irreversible production decision. The detector identified deviations from normal appearance, although the governance layer determined how evidence was handled within the prototype evaluation procedure.

An operator's review could subsequently modify the final disposition. For example, a record initially routed for review could be marked as a false-positive detection after

examination of its supporting images. The system then changed the recommendation to *RELEASE_AFTER_REVIEW*. A confirmed defect could instead remain on hold or be assigned a reject-related action.

The capsule demonstration provided a representative example. The selected record was initially routed for review because its fused confidence was below the operational threshold, and its semantic label remained unknown. After the operator inspected the annotated image, heatmap, predicted mask, and anomaly map, the case was recorded as a false-positive release. The final quality assurance action was consequently changed to *RELEASE_AFTER_REVIEW*.

This behaviour highlights the importance of separating anomaly detection from quality dispositions. A visibly unusual region is not necessarily a defect of production to be rejected. Preserving the original detector response and storing the operator's decision separately meant the audit trail contained both the model evidence and the human judgement.

4.10 PostgreSQL audit database validation

For persistent and queryable storage of the inspection results, PostgreSQL was employed. The completed database contains 650 inspection records from the experiments of PatchCore and PaDiM on bottle, capsule, and hazelnut.

Table 4.6 summarises the number of stored records and those requiring operator review.

Table 4.6: PostgreSQL inspection-record and review summary

Category	Model	Inspection records	Requires review	Review rate
Bottle	PaDiM	83	57	0.6867
Bottle	PatchCore	83	16	0.1928
Capsule	PaDiM	132	111	0.8409
Capsule	PatchCore	132	68	0.5152
Hazelnut	PaDiM	110	99	0.9000
Hazelnut	PatchCore	110	19	0.1727
Total	All models	650	370	0.5692

The database totals were consistent with individual experiment results. PatchCore produced substantially fewer review-required records than PaDiM in each category. The difference was smallest for capsule, which remained the most demanding category for both models, and largest for hazelnut, where 90.00% of the PaDiM records required review compared with 17.27% for PatchCore.

The overall review rate was approximately 56.92%, although this aggregate value combines both models and should not be interpreted as the review workload of the selected final configuration. The more relevant operational result was that PatchCore reduced review demand consistently across the evaluated categories.

The database schema preserved the principal inspection fields, including the image identifier, product category, model name, semantic label, risk class, fused confidence and

review requirement. Additional localisation, confidence and configuration evidence were retained within a structured JSONB payload.

Operator reviews were stored in a related table linked to the original inspection record by a foreign key. This allowed the initial automated assessment and subsequent human decision to remain distinct while still forming one traceable inspection history.

The PostgreSQL service was deployed through Docker Compose. Service-status checks confirmed that the PostgreSQL 16 container was running and healthy on the configured host port. The successful persistence and retrieval of all 650 records showed that the audit layer was functioning as intended.

```
• (.venv) PS C:\Users\USER\Documents\Dissertation\risk_aware_inspection_prototype> docker compose ps
NAME                IMAGE             COMMAND           SERVICE   CREATED   STATUS    PORTS
risk_inspection_postgres postgres:16       "docker-entrypoint.s..." postgres  13 days ago Up 37 minutes (healthy) 0.0.0.0:55432->5432/tcp, [::]:55432->5432/tcp
○ (.venv) PS C:\Users\USER\Documents\Dissertation\risk_aware_inspection_prototype>
```

Figure 4.2: Docker Compose service status confirming the healthy PostgreSQL audit database.

4.11 FastAPI and NiceGUI operational validation

4.11.1 FastAPI backend

The FastAPI backend offered the database and decision-support functionality through a well-defined set of HTTP endpoints. These were: health checking, record retrieval, individual record lookup, QA-decision retrieval, operator review submission, and database summary reporting.

The /health endpoint returned a successful status, confirmed the PostgreSQL connection and reported the presence of 650 inspection records. This provided a direct operational check that the backend and database were communicating correctly.

The automatically generated OpenAPI interface displayed the implemented routes:

- GET /health;
- GET /records;
- GET /records/{record_id};
- GET /records/{record_id}/qa-decision;
- POST /reviews;
- GET /summary.

The API documentation indicated that the functional operations were not exclusively integrated into the graphical interface. They were exposed via a reusable backend framework capable of supporting an alternative dashboard, external client, or future integration with a manufacturing information system.



Figure 4.3: Automatically generated OpenAPI documentation for the risk-aware inspection backend.

4.11.2 NiceGUI dashboard

The NiceGUI dashboard provided the operator-facing layer of the prototype. It connected to the same PostgreSQL audit database and displayed the number of records and review-required cases for each product category and model.

The summary view showed all 650 inspection records. Its values agreed with the database queries and experiment results, providing evidence that the graphical interface was retrieving stored records rather than displaying manually entered values.

The dashboard also allowed an individual record to be loaded using its database identifier. Once selected, the interface displayed the image identifier, product category, anomaly model, semantic label, risk class, fused confidence, review requirement and reasons for escalation.

The quality-assurance section presented the current disposition and explained whether further operator action was required. Where a review had already been saved, the latest operator decision, reviewer identifier and note were displayed alongside the automated recommendation.

The completed interface therefore provided more than a record viewer. It connected anomaly evidence, governance rules, and operator decisions within a single inspection workflow.

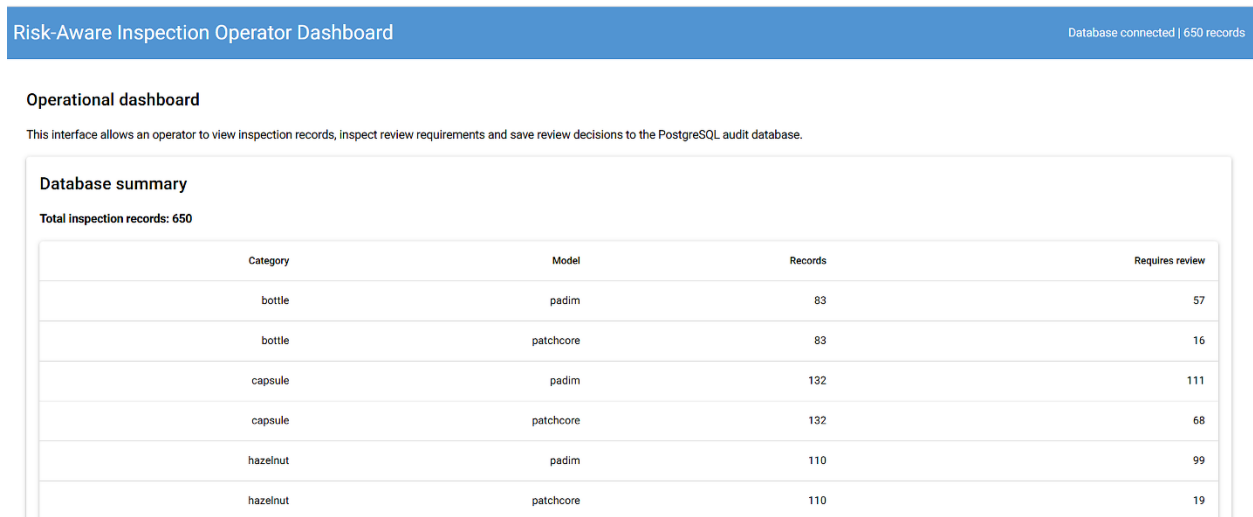


Figure 4.4: NiceGUI dashboard showing the PostgreSQL inspection-record summary by category and anomaly detection model.

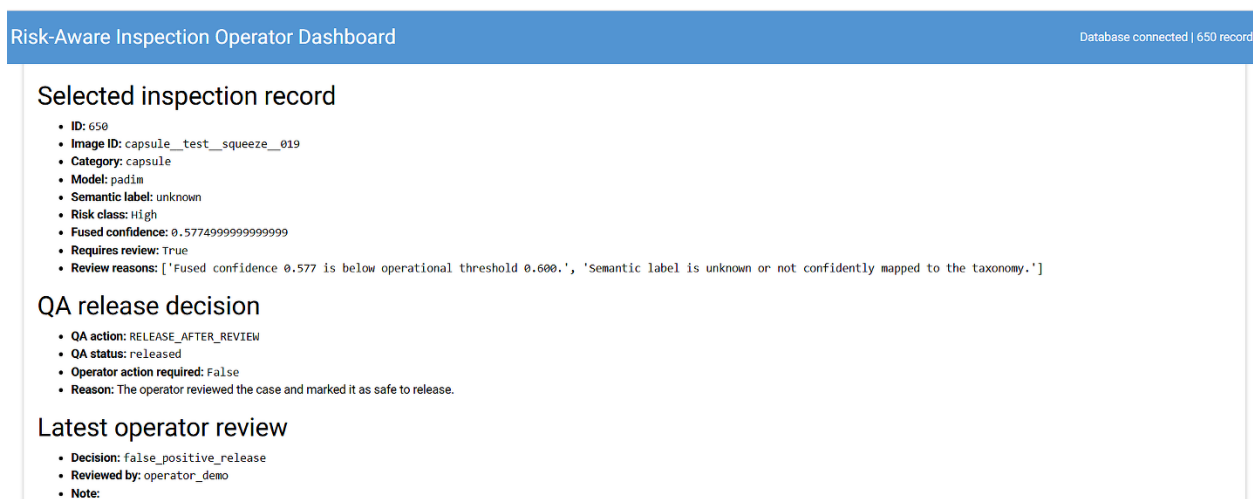


Figure 4.5: Selected capsule inspection record showing the risk-governed QA disposition and latest operator review.

4.12 Evidence-linked operator review

The evidence-linked review panel was introduced to ensure that an operator did not have to make a decision on numerical scores alone. The dashboard found the visual artefacts produced by the pipeline for the given record and displayed them.

The displayed evidence included:

- the annotated inspection image;
- the anomaly heatmap overlay;
- the predicted binary mask;
- the continuous anomaly map.

The annotated image highlighted the region selected by the localisation stage and showed confidence in this region. The heatmap overlay showed the spatial distribution of the anomaly response over the original image. The predicted mask showed the region after thresholding and morphological processing. The anomaly map showed the underlying continuous detector response.

The capsule example showed that the main anomaly response was localised to one end of the product. The operator could determine from the combined view of the different representations whether the highlighted area represented a meaningful production defect, a benign visual variation, or an overly sensitive model response.

This evidence was of particular importance in addressing false positives. The operator might view the model response and then decide whether to release the item as a false positive, confirm a defect, or escalate it for engineering review. After being saved, the decision was linked with the original database record and affected the final QA disposition.

The evidence panel therefore established a traceable sequence from model prediction to human decision:

- I. The detector generated an anomaly score and map

- II. The localisation stage identified a candidate region
 - III. The semantic and risk stages determined whether review was required
 - IV. The dashboard displayed the supporting visual evidence
 - V. The operator recorded a decision
 - VI. The decision was saved in PostgreSQL
 - VII. The QA disposition was recalculated.
- This functionality rectified a significant constraint of entirely automated inspection. Instead of delivering an unexplained pass or fail result, the prototype exposed the evidence used to support the recommendation.

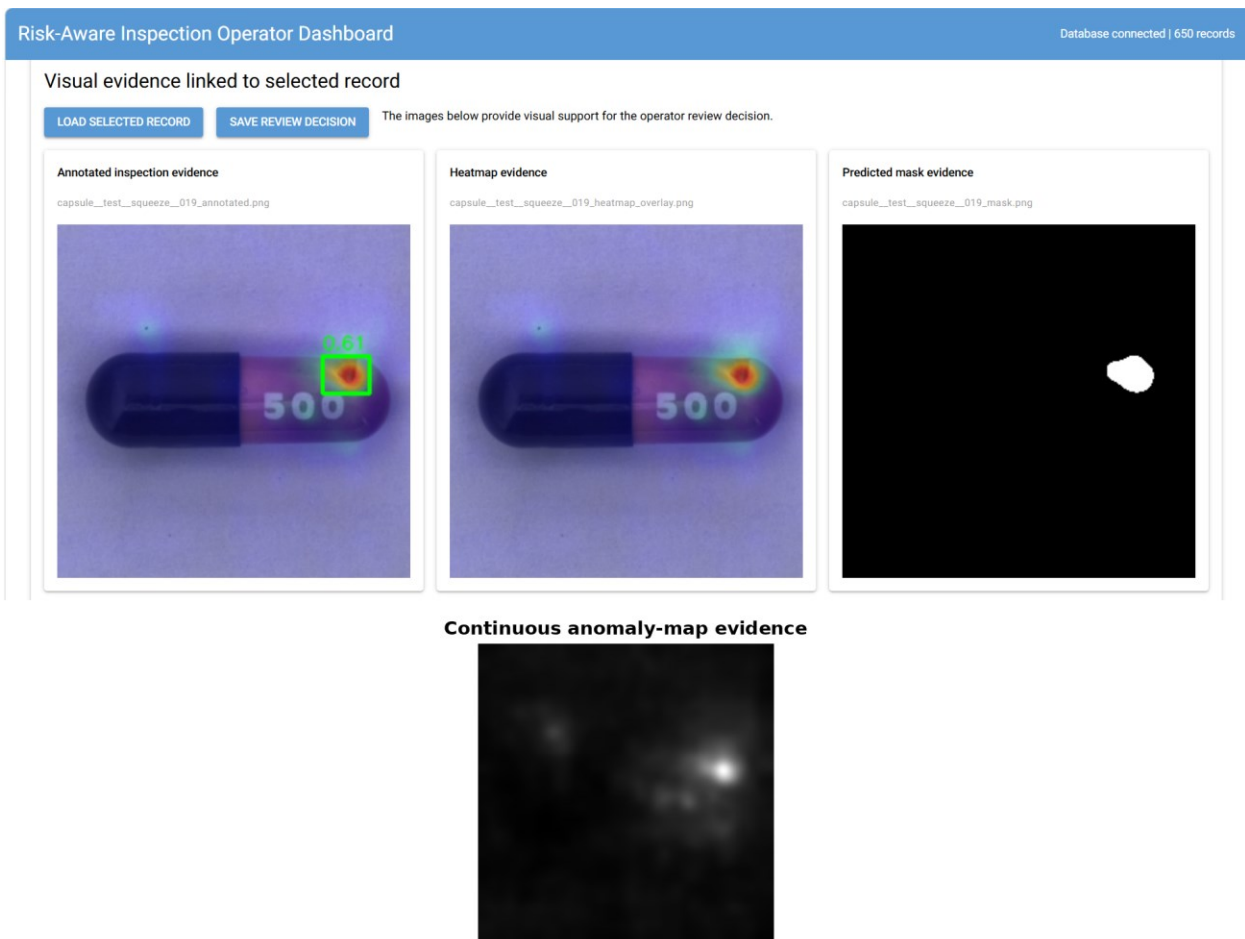


Figure 4.6: Evidence-linked operator review panel showing annotated, heatmap, mask and anomaly-map evidence for a capsule inspection record.

4.13 Scalability and operational relevance

The prototype showed that the selected PatchCore detector (Roth et al., 2022) could be embedded in a modular quality-control architecture rather than being a stand-alone model. The analytical, decision-support, storage, service and interface layers were separated, allowing individual components to be modified without the need to re-design the entire workflow. For example, the detector may be exported or accelerated later, while the NiceGUI dashboard could be replaced with an industrial human-machine interface without removing the FastAPI, audit and operator review functions.

At the data and deployment level, indexed PostgreSQL fields supported filtering by product category, detector, risk class and review status, while the JSONB payload retained supplementary evidence without requiring a new relational column for every experimental output. Docker Compose provided a consistent boundary between the PostgreSQL service and the Python applications. These features support expansion to additional products and longer audit histories, although database load, concurrent access and evidence-retention performance would require validation under production conditions.

The experiments were completed on CPU-based hardware comprising an Intel Core i5-8250U processor and approximately 8 GB of RAM. This demonstrated that the framework could be evaluated without dedicated GPU hardware, but the recorded experimental durations should not be interpreted as production-line throughput. Deployment would require benchmarking against the intended camera resolution, inspection cycle time, maximum response latency and record volume. GPU acceleration, model export, batch optimisation or edge-computing hardware may be necessary for continuous high-speed inspection.

The architecture comparison also informed practical model selection. ResNet50 was retained because WideResNet50-2 did not provide a consistent performance improvement across the three categories. Avoiding unsupported additional capacity reduced computational demand while preserving the best overall balance observed in the experiments. The completed prototype therefore provides an extensible and auditable basis for further industrial validation, rather than evidence that the present implementation is already production ready.

4.14 Limitations

Several limitations should be considered when interpreting the results.

First, the quantitative evaluation was conducted on three MVTec AD categories exclusively. Bottle, capsule and hazelnut contributed useful variation in product geometry and defect appearance, yet they are not representative of all industrial inspection conditions. The findings cannot therefore be immediately extrapolated to other materials, production techniques, or camera arrangements.

Second, the studies were performed on a public benchmark, not on images taken from a live production line. MVTec AD offers controlled and reproducible data, but a real-world manufacturing environment can have variability in illumination, vibration, product orientation, lens contamination, and background appearance.

Third, the architecture and the threshold comparisons were implemented on a CPU-only workstation. This demonstrated computational feasibility, but it limited the scope of latency and throughput tests. The current prototype was not intended to satisfy real-time industrial timing constraints.

Fourth, the localisation threshold optimisation used the available ground-truth masks to identify the best threshold for each category and model. These results suggest the potential value of the anomaly maps, but they may be optimistic if the same samples were used to select the threshold and report performance. In a larger study, there would be distinct calibration and test sets.

Fifth, the semantic interpretation stage used engineered visual evidence and constrained taxonomy. This allowed for traceability but did not provide the detailed defect recognition

capability of a separately trained semantic classifier. The category-specific fallback rules would need more validation before being transferred to new product types.

Sixth, the risk matrix was a prototype policy rather than a formally validated Failure Modes and Effects Analysis. Domain specialists would need to review the assigned risk classes and disposition rules before operational deployment.

Seventh, there was no formal user study, but controlled review cases were used to demonstrate operator interaction. The interface was shown to function technically, but usability, decision time, reviewer consistency, and operator trust were not measured experimentally.

Finally, the prototype did not control physical production equipment. The outputs were database records and recommendations. Integration with a conveyor, reject mechanism, programmable logic controller or manufacturing execution system would require further validation for safety, cybersecurity and failure-recovery.

These limitations do not invalidate the reported results but set the scope for understanding the conclusions.

Chapter 5: Conclusion and Recommendations

5.1 Summary of the main project

This dissertation investigated how normal-only visual anomaly detection could be extended from model output into risk-aware industrial inspection decision support. The central argument was that an anomaly score alone is insufficient for a complete quality decision. A useful inspection workflow must also indicate where the anomaly occurs, whether the visual evidence supports an approved defect description, how confident the combined evidence is, what risk the case presents, and whether the outcome can be resolved automatically or requires operator review.

The evaluation used the bottle, capsule, and hazelnut categories from MVTec AD, which offer normal training images, anomalous test images, and pixel-level annotations (Bergmann et al., 2019). PatchCore and PaDiM were selected because both support normal-only training but represent normal appearance differently: PatchCore uses retained normal patch embeddings, whereas PaDiM models position-specific feature distributions (Defard et al., 2021; Roth et al., 2022). The architecture comparison identified PatchCore with ResNet50 as the most consistent image-level configuration. It achieved AUROC and F1-score values of 1.0000 and 0.9920 for bottle, 0.9936 and 0.9818 for capsule, and 1.0000 and 0.9928 for hazelnut.

The localisation results showed why detector selection could not be based on image-level performance alone. With the original thresholds, PaDiM achieved higher mean IoU and Dice scores for bottle (0.4968 and 0.6442), hazelnut (0.3084 and 0.4444) and capsule (0.1731 and 0.2673). After category-specific threshold optimisation, PatchCore produced the strongest mean IoU and Dice results: 0.5929 and 0.7322 for bottle, 0.4331 and 0.5854

for hazelnut, and 0.2979 and 0.4311 for capsule. The improvement confirmed that continuous anomaly maps contained useful spatial evidence that was not fully captured by the default binary thresholds.

Operational evaluation further distinguished the detectors. PatchCore required review for 19.28% of bottle records, 17.27% of hazelnut records and 51.52% of capsule records, compared with 68.67%, 90.00% and 84.09% respectively for PaDiM. The capsule semantic fallback reduced the PatchCore review rate from 68.18% to 51.52% and the PaDiM rate from 87.12% to 84.09%, without changing their integrated F1-scores. These findings supported PatchCore with ResNet50 as the preferred detector while retaining PaDiM as an important statistical comparison and a useful reference for localisation behaviour.

5.2 Unique contribution of the project

The principal contribution is not a new anomaly-detection algorithm. It is the design and evaluation of a prototype that connects established normal-only detectors to an accountable quality-decision workflow. Detection, localisation, constrained semantic interpretation, confidence fusion and deterministic risk assessment were evaluated as connected stages rather than isolated technical outputs. This made it possible to examine the consequences of model behaviour for review workload and final quality disposition.

A second contribution is the distinction between image-level detection and spatial usefulness. The comparison demonstrated that a detector with strong AUROC and F1-score may still require separate threshold calibration to obtain effective masks, while a detector with comparatively strong default localisation may generate an impractical review burden. Evaluating classification, IoU, Dice, semantic uncertainty and review rates

together therefore provided a more operationally meaningful basis for model selection than ranking architectures using a single metric.

The project also demonstrated evidence-linked human oversight. Each inspection record retained the model result, localisation evidence, semantic label, confidence values, risk class, reason for review and operator decision. PostgreSQL provided persistent audit storage, FastAPI exposed the inspection and review functions, and the NiceGUI dashboard linked records to annotated images, heatmaps, predicted masks and continuous anomaly maps. This allowed an operator to examine the available evidence before confirming, releasing or escalating a case.

Thus, the detector was not permitted to independently provide an irreversible production decision. Instances with low confidence, unknown semantic evidence, incomplete risk information, or high risk were referred for human review. The importance of validity, accountability, transparency, and human oversight in AI risk management is reflected in this architecture (NIST, 2023). The resulting contribution is therefore a traceable risk-aware inspection framework that demonstrates how anomaly evidence can support, rather than replace, manufacturing quality judgement.

5.3 Recommendations for future work

The first priority is evaluation using images collected from real industrial environments. Although MVTec AD provided a reproducible benchmark, live production data would introduce changes in illumination, product position, vibration, background appearance, lens condition and product drift. Further work should include additional product categories, multiple acquisition periods and unseen production batches so that robustness and transferability can be assessed beyond the controlled benchmark.

Threshold calibration should be separated from final testing. The current optimisation used available ground-truth masks to identify category- and detector-specific thresholds, which may produce optimistic estimates when the same data inform selection and evaluation. A dedicated calibration set should be used to select thresholds, followed by testing on previously unseen images. External validation on another dataset or production site would provide stronger evidence of generalisability.

The framework should also be benchmarked against genuine operational timing requirements. The present experiments were completed on an Intel Core i5-8250U CPU with approximately 8 GB of RAM and were not intended to establish production-line throughput. Future evaluation should measure end-to-end latency, camera-resolution effects, sustained record volume and concurrent review activity. Model export, GPU acceleration, batch optimisation or edge hardware could then be assessed against a defined inspection cycle rather than adopted without measured need.

Confidence fusion, semantic interpretation and risk policy require further validation. Formal confidence-calibration methods should be compared with the existing transparent rule-based approach, with attention to missed defects and unnecessary review. Manufacturing specialists should refine the defect taxonomy and conduct a product-specific Failure Modes and Effects Analysis before the risk classes or disposition rules are used operationally. This would preserve interpretability while grounding the decision policy in domain evidence.

A structured user study is needed to assess the evidence-linked dashboard. Relevant measures include review time, decision consistency, the clarity of visual evidence, operator trust and disagreement between reviewers. Comparing the complete evidence

panel with score-only or mask-only interfaces would show whether the proposed presentation genuinely improves human decision-making and whether additional training or interface changes are necessary.

Finally, integration with manufacturing execution systems, programmable logic controllers or robotic handling equipment should occur only after appropriate safety, cybersecurity, latency and failure-recovery validation. Long-term monitoring should also identify changes in input data, model confidence, review workload and operator decisions. These steps would provide the evidence required to progress from a laboratory decision-support prototype towards a production-ready inspection system.

5.4 Conclusion

This study has shown that normal-only anomaly detection is a strong basis for industrial visual inspection, but not a complete quality-control solution. PatchCore with ResNet50 provided the strongest overall image-level and operational balance, while PaDiM showed why localisation must be assessed separately. The broader contribution is a risk-aware and auditable framework that uses localisation, constrained semantic interpretation, confidence handling, risk rules, human-review gating and persistent evidence to prevent uncertain or consequential cases from being treated as final decisions. Despite its benchmark, CPU and prototype-policy limitations, the project provides a credible foundation for accountable industrial anomaly detection and traceable manufacturing quality decisions.

References

- Akcay, S., Ameln, D., Vaidya, A., Lakshmanan, B., Ahuja, N. and Genc, U. (2022) 'Anomalib: A deep learning library for anomaly detection', 2022 IEEE International Conference on Image Processing (ICIP), pp. 1706–1710. doi:10.1109/ICIP46576.2022.9897283.
- Amershi, S., Weld, D., Vorvoreanu, M., Fournery, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P.N., Inkpen, K., Teevan, J., Kikin-Gil, R. and Horvitz, E. (2019) 'Guidelines for human-AI interaction', Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, paper 3, pp. 1–13. doi:10.1145/3290605.3300233.
- Bergmann, P., Fauser, M., Sattlegger, D. and Steger, C. (2019) 'MVTec AD—A comprehensive real-world dataset for unsupervised anomaly detection', Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 9592–9600. doi:10.1109/CVPR.2019.00982.
- Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D. and Steger, C. (2022) 'Beyond dents and scratches: Logical constraints in unsupervised anomaly detection and localization', International Journal of Computer Vision, 130(4), pp. 947–969. doi:10.1007/s11263-022-01578-9.
- Czimmermann, T., Ciuti, G., Milazzo, M., Chiurazzi, M., Roccella, S., Oddo, C.M. and Dario, P. (2020) 'Visual-based defect detection and classification approaches for industrial applications—A survey', Sensors, 20(5), article 1459. doi:10.3390/s20051459.
- Defard, T., Setkov, A., Loesch, A. and Audigier, R. (2021) 'PaDiM: A patch distribution modeling framework for anomaly detection and localization', in Del Bimbo, A. et al. (eds.)

Pattern Recognition: ICPR International Workshops and Challenges. Lecture Notes in Computer Science, vol. 12664. Cham: Springer, pp. 475–489. doi:10.1007/978-3-030-68799-1_35.

Dice, L.R. (1945) ‘Measures of the amount of ecologic association between species’, *Ecology*, 26(3), pp. 297–302. doi:10.2307/1932409.

Fawcett, T. (2006) ‘An Introduction to ROC Analysis’, *Pattern Recognition Letters*, 27(8), pp. 861–874. doi:10.1016/j.patrec.2005.10.010.

He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. doi:10.1109/CVPR.2016.90.

International Electrotechnical Commission (2018) IEC 60812:2018 Failure modes and effects analysis (FMEA and FMECA). Geneva: International Electrotechnical Commission.

Liu, J., Xie, G., Wang, J., Li, S., Wang, C., Zheng, F. and Jin, Y. (2024) ‘Deep industrial image anomaly detection: A survey’, *Machine Intelligence Research*, 21(1), pp. 104–135. doi:10.1007/s11633-023-1459-z.

National Institute of Standards and Technology (NIST) (2023) Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1. Gaithersburg, MD: U.S. Department of Commerce. doi:10.6028/NIST.AI.100-1.

Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T. and Gehler, P. (2022) ‘Towards total recall in industrial anomaly detection’, *Proceedings of the IEEE/CVF Conference on*

Computer Vision and Pattern Recognition, pp. 14318–14328.
doi:10.1109/CVPR52688.2022.01392.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F. and Dennison, D. (2015) ‘Hidden technical debt in machine learning systems’, *Advances in Neural Information Processing Systems*, 28, pp. 2503–2511.

Tao, X., Gong, X., Zhang, X., Yan, S. and Adak, C. (2022) ‘Deep learning for unsupervised anomaly localization in industrial images: A survey’, *IEEE Transactions on Instrumentation and Measurement*, 71, article 5018021, pp. 1–21.
doi:10.1109/TIM.2022.3196436.

Appendices

Appendix A: Selected core framework excerpts

This appendix presents five selected code excerpts that evidence the core functions of the risk-aware inspection framework: anomaly detection, localisation evaluation, constrained semantic interpretation, risk-aware human-review gating, and audit-record persistence. The selection is intentionally limited to avoid reproducing the complete implementation. The comparative PaDiM, confidence-fusion, risk-matrix, FastAPI and NiceGUI modules remain available in the version-controlled repository identified in Appendix D.

A.1 PatchCore anomaly-detection experiment

Source file: *scripts\train_patchcore_folder_mvtec.py*

```
datamodule = Folder(
    name=category_root.name,
    root=category_root,
    normal_dir="train/good",
    normal_test_dir="test/good",
    abnormal_dir=[str(path.relative_to(category_root)) for path in abnormal_dirs],
    train_batch_size=train_batch_size,
    eval_batch_size=eval_batch_size,
    num_workers=num_workers,
    val_split_mode="same_as_test",
)

model = Patchcore(
    backbone=backbone,
    layers=("layer2", "layer3"),
    pre_trained=True,
    coreset_sampling_ratio=coreset_sampling_ratio,
    num_neighbors=num_neighbors,
)

engine = Engine(
    default_root_dir=output_dir,
    accelerator="cpu",
    devices=1,
    max_epochs=1,
)

print("Starting PatchCore Folder experiment")
print(f"Category root: {category_root}")
print(f"Output directory: {output_dir}")
print("Abnormal folders:")
for path in abnormal_dirs:
    print(f"    {path.name}")

engine.fit(model=model, datamodule=datamodule)
engine.test(model=model, datamodule=datamodule)

print("PatchCore Folder experiment complete.")
```

This code configured and executed the PatchCore experiments across all selected MVTec AD categories. Normal training images were obtained from the train/good folder, while the normal and defective test folders were supplied through the Anomalib Folder data module. PatchCore employed intermediate features from layer2 and layer3, featuring configurable backbone, coreset sampling ratio, and nearest-neighbor settings. The Anomalib Engine then fitted and tested the detector using CPU execution, conforming to the experimental environment described in the methodology.

A.2 Pixel-level localisation metrics

Source file: scripts\evaluate_localisation_mvtec.py

```
def compute_iou_and_dice(pred: np.ndarray, gt: np.ndarray) -> tuple[float, float]:
    pred = pred.astype(bool)
    gt = gt.astype(bool)

    intersection = np.logical_and(pred, gt).sum()
    union = np.logical_or(pred, gt).sum()
    pred_sum = pred.sum()
    gt_sum = gt.sum()

    iou = intersection / union if union > 0 else 1.0
    dice = (2 * intersection) / (pred_sum + gt_sum) if (pred_sum + gt_sum) > 0 else 1.0

    return float(iou), float(dice)
```

This function calculated the pixel-level Intersection over Union and Dice coefficient used to evaluate predicted anomaly masks in comparison to the MVTec AD ground-truth masks. Both masks were converted to Boolean arrays before their intersection, union and areas were measured. The resulting IoU and Dice values constituted the basis for the localisation comparison and subsequent threshold-optimisation experiments reported in Chapter 4.

A.3 Constrained semantic interpretation

Source file: `src\risk_aware_inspection\semantics.py`

```
20.0))
    elongated = box.aspect_ratio >= float(self.rule_cfg.get("elongated_aspect_ratio", 3.2))
    thin_region = box.area_ratio <= float(self.rule_cfg.get("thin_area_ratio_max", 0.025))
    dark = dark_contrast >= float(self.rule_cfg.get("dark_contrast_threshold", 18.0))
    saturated = saturation_contrast >= float(self.rule_cfg.get("saturation_contrast_threshold",
    large = box.area_ratio >= float(self.rule_cfg.get("large_region_ratio", 0.08))
    edge_rich = edge_density >= float(self.rule_cfg.get("edge_density_threshold", 0.06))

    label = self.unknown_label
    rule_strength = 0.0
    evidence_rules: list[str] = []

    if elongated and thin_region and edge_rich and dark:
        label = "crack"
        rule_strength = 0.92
        evidence_rules.append("elongated_thin_dark_edge_rich_crack_like")
    elif elongated and thin_region:
        label = "scratch"
        rule_strength = 0.78 + 0.10 * float(edge_rich)
        evidence_rules.append("elongated_thin_region")
    elif edge_rich and dark:
        label = "crack"
        rule_strength = 0.74 + 0.08 * float(elongated) + 0.08 * float(thin_region)
        evidence_rules.append("dark_edge_rich_crack_like_region")
    elif saturated:
        label = "contamination"
        rule_strength = 0.74 + clamp(saturation_contrast / 80.0) * 0.18
        evidence_rules.append("colour_or_saturation_shift")
    elif large and not edge_rich:
        label = "stain"
        rule_strength = 0.70
        evidence_rules.append("large_low_edge_region")
    elif dark and not elongated:
        label = "dent"
        rule_strength = 0.68 + clamp(dark_contrast / 100.0) * 0.20
        evidence_rules.append("local_dark_depression_like_region")
    elif large and edge_rich:
        label = "deformation"
        rule_strength = 0.66
        evidence_rules.append("large_irregular_edge_region")
    else:
        if box.max_heat >= 0.90 and box.area_ratio >= 0.02:
            label = "surface_defect"
            rule_strength = 0.62
            evidence_rules.append("high_heat_moderate_area_surface_defect_fallback")
        else:
            label = self.unknown_label
            rule_strength = float(self.rule_cfg.get("minimum_semantic_confidence", 0.35))
            evidence_rules.append("no_rule_matched_confidently")

    label = self._safe_label(label)
    confidence = clamp(0.65 * rule_strength + 0.35 * box.max_heat)
    return SemanticResult(
        label=label,
        confidence=confidence,
        method="rules_constrained_taxonomy",
        evidence={
            "selected_box": box.to_dict(),
            "edge_density": edge_density,
            "mean_saturation": mean_saturation,
            "mean_value": mean_value,
            "saturation_contrast": saturation_contrast,
            "dark_contrast": dark_contrast,
            "rules": evidence_rules,
        },
    )
```

This block performed the constrained semantic interpretation stage applied on a localised anomalous region. Labels (crack, scratch, contamination, stain, dent, deformation or surface_defect) were associated with evidence using deterministic criteria based on geometric, intensity, edge and colour characteristics. Where no rule matched sufficiently, the output remained unknown instead of being labelled an unsupported defect. The semantic confidence combines the rule strength with the maximum anomaly response in the selected region.

A.4 Risk-aware human-review gating

Source file: `src\risk_aware_inspection\gating.py`

```
def decide_review(
    *,
    detection: DetectionResult,
    localisation: LocalisationResult,
    semantic: SemanticResult,
    risk: RiskResult,
    fused_confidence: float,
    cfg: dict,
) -> ReviewDecision:
    """Apply human-review escalation rules for uncertainty, contradiction and unmapped risk
    cases."""
    reasons: list[str] = []

    if fused_confidence < float(cfg.get("minimum_fused_confidence", 0.60)):
        reasons.append(
            f"Fused confidence {fused_confidence:.3f} is below operational threshold "
            f"{float(cfg.get('minimum_fused_confidence', 0.60)):.3f}."
        )

    if cfg.get("force_review_for_unknown_label", True) and semantic.label == "unknown":
        reasons.append("Semantic label is unknown or not confidently mapped to the taxonomy.")

    if cfg.get("force_review_for_unmapped_rpm", True) and not risk.mapped:
        reasons.append(f"RPM lookup tuple {risk.rpm_lookup_key} is not mapped in the explicit
        table.")

    if (
        cfg.get("force_review_when_no_localisation_for_anomaly", True)
        and detection.is_anomalous
        and localisation.component_count == 0
    ):
        reasons.append("Image-level anomaly was detected but no stable localised region was
        extracted.")

    near_margin = float(cfg.get("near_boundary_margin", 0.07))
    if detection.score_margin <= near_margin:
        reasons.append(
            f"Anomaly score is close to the decision boundary;
            margin={detection.score_margin:.3f}."
        )

    if detection.is_anomalous and semantic.label == "normal":
        reasons.append("Contradiction: detector marked anomaly but semantic stage returned
        normal.")

    if risk.risk_class == "Critical":
        reasons.append("Critical risk class requires operator confirmation before release
        decision.")

    return ReviewDecision(requires_review=bool(reasons), reasons=reasons)
```

This function applied the final human-review gate to fused confidence, semantic evidence, localisation status and risk classification. Reasons for escalation included low confidence, an unknown semantic label, an unmapped risk tuple, failed localisation, proximity to the detector decision boundary, contradictory evidence and a critical risk class. Returning both the review requirement and its explicit reasons preserved the traceable link between the upstream confidence and risk logic and the operator-review workflow.

A.5 PostgreSQL audit-record persistence

Source file: `src\risk_aware_inspection\audit_db.py`

```
def insert_inspection_record(connection, record: dict[str, Any]) -> int:
    """Insert one inspection record and return its database ID."""
    query = """
        INSERT INTO inspection_records (
            image_id,
            image_path,
            category,
            model_name,
            anomaly_score,
            anomaly_threshold,
            is_anomalous,
            semantic_label,
            semantic_confidence,
            risk_class,
            fused_confidence,
            requires_review,
            review_reasons,
            result_payload
        )
        VALUES (
            %(image_id)s,
            %(image_path)s,
            %(category)s,
            %(model_name)s,
            %(anomaly_score)s,
            %(anomaly_threshold)s,
            %(is_anomalous)s,
            %(semantic_label)s,
            %(semantic_confidence)s,
            %(risk_class)s,
            %(fused_confidence)s,
            %(requires_review)s,
            %(review_reasons)s,
            %(result_payload)s
        )
        RETURNING id;
    """

    prepared_record = dict(record)
    prepared_record["result_payload"] = json.dumps(prepared_record["result_payload"])

    with connection.cursor() as cursor:
        cursor.execute(query, prepared_record)
        inserted = cursor.fetchone()

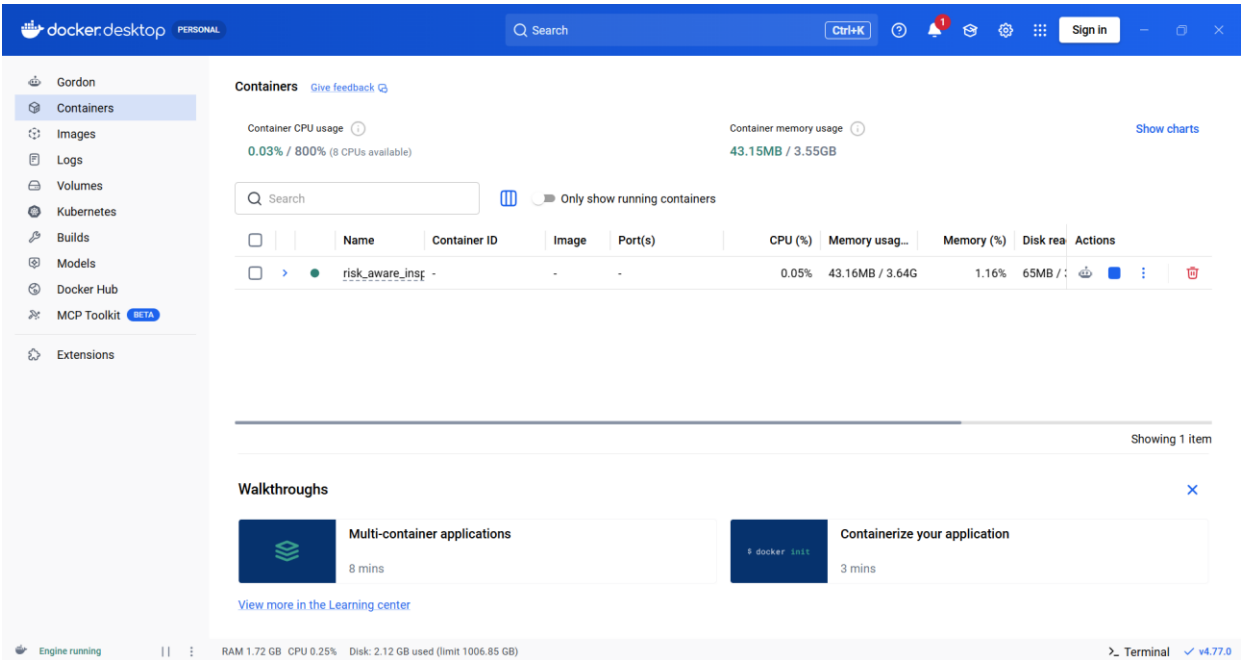
    connection.commit()
    return int(inserted["id"])
```

This function stored each completed inspection result in the PostgreSQL audit database. The database record retained the detector output, semantic interpretation, risk class, fused confidence, review requirement, and review reasons, whilst `result_payload` preserved the broader evidence in JSON format. The returned database identifier provided the link used by later operator-review records, enabling traceability from automated inspection to human decision.

Appendix B: Supplementary operational evidence

Appendix B presents supplementary evidence from the executed operational inspection framework. The images indicate that the results of anomaly detection were linked to a containerised audit database, backend services, and an interface for operator review. The material complements the implementation and validation outlined in Chapter 4 by recording the key software elements used to inspect, review, and preserve anomaly evidence.

B.1 PostgreSQL service running through Docker desktop

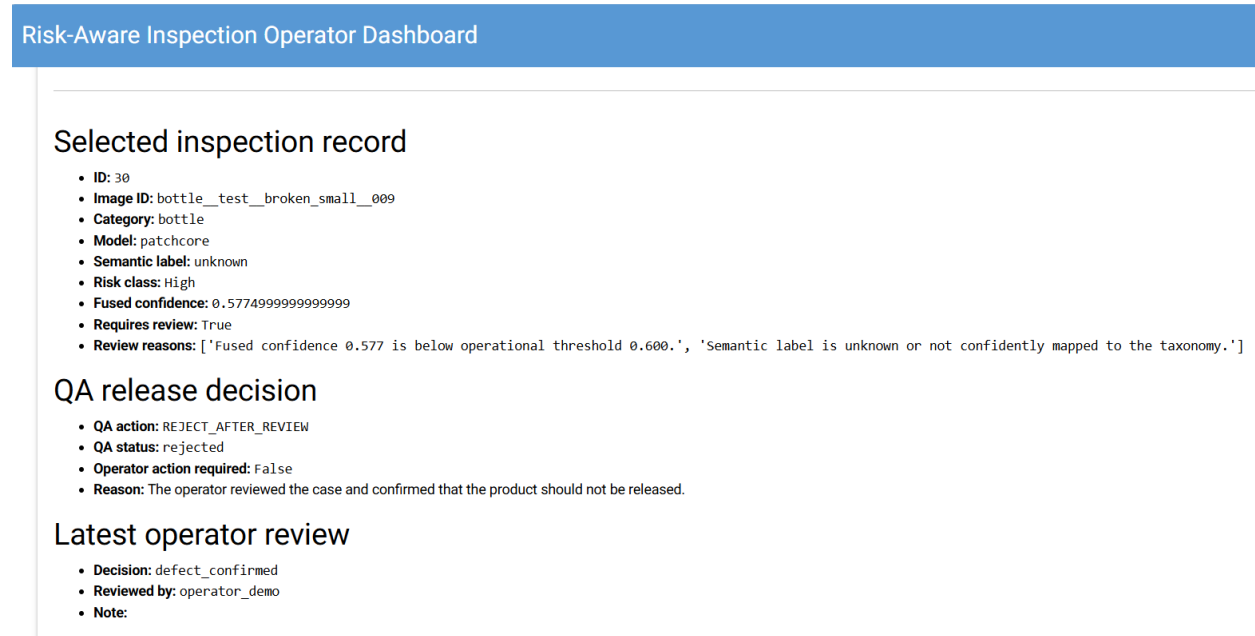


Appendix Figure B.1: Docker Desktop showing the containerised PostgreSQL service used by the risk-aware inspection prototype.

The PostgreSQL audit service was run as a Docker-managed container so that database operation remained separate from the Python environments used for anomaly detection and the operator interface. This supported repeatable service startup and reduced

dependence on a machine-specific PostgreSQL installation. The database provided persistent storage for inspection records, review reasons and operator decisions.

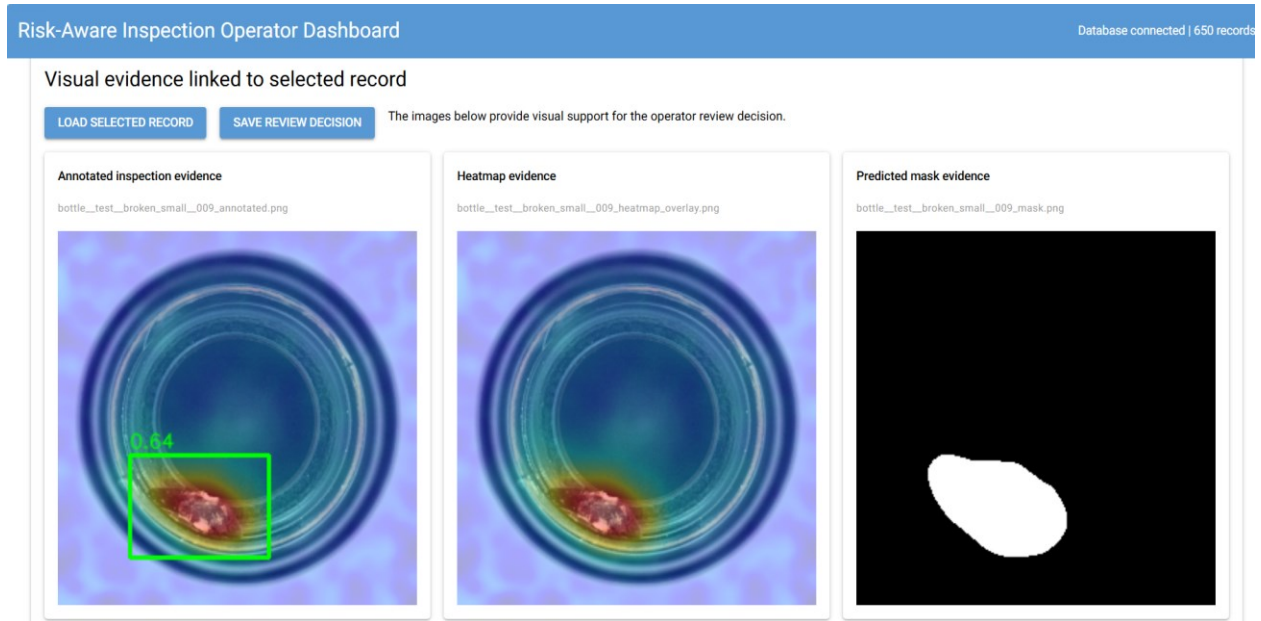
B.2 Selected inspection record and QA disposition



Appendix Figure B.2: Bottle inspection record showing the automated assessment, review reasons, QA disposition and latest operator review.

This bottle example supplements the capsule record presented in Chapter 4. It shows the detector output together with the semantic label, risk class, fused confidence, review requirement and explicit reasons for escalation. The QA section records the current disposition and retains the operator decision alongside the automated recommendation, preserving the distinction between model evidence and human judgement.

B.3 Evidence-linked operator review



Appendix Figure B.3: Evidence-linked bottle review panel showing the annotated image, heatmap and predicted mask for the selected inspection case.

The evidence-linked panel associates the selected bottle inspection record with the visual artefacts produced by the anomaly-detection and localisation stages. Presenting the annotated image, heatmap and binary mask together allows the reviewer to inspect the basis of the automated recommendation before recording a decision. This figure therefore extends, rather than repeats, the capsule evidence example used in Chapter 4.

Appendix C: Experimental and computational configuration

Chapter 3 reports the computational environment, dataset organisation, evaluated backbones and principal experimental controls. This appendix therefore retains only script-level settings not tabulated fully in the main methodology, including feature layers, detector-specific batch sizes, worker count and validation-split mode.

Appendix Table C.1: Supplementary implementation parameters for PatchCore and PaDiM.

Implementation parameter	PatchCore	PaDiM
Feature layers	layer2, layer3	layer1, layer2, layer3
Pre-trained backbone	Yes	Yes
Coreset sampling ratio	0.1 script default	Not applicable
Nearest neighbours	9 script default	Not applicable
Retained features	Not applicable	Default for ResNet18; 100 for tested ResNet50 configuration
Default training batch size	8	4
Default evaluation batch size	8	4
Number of workers	0	0
Validation split mode	same_as_test	same_as_test

These settings document the detector-specific defaults used by the project scripts. The architecture comparison and rationale for selecting PatchCore with ResNet50 remain in Chapters 3 and 4; the table above is retained only as supplementary reproducibility information.

Appendix D: Reproducibility and repository evidence

Appendix D shows the primary local execution sequence and repository information necessary for replicating the operational prototype. The complete source code implementation is retained in the version-controlled project repository, while the commands below specify the principal services needed to reproduce the PostgreSQL-backed API and NiceGUI operator dashboard.

D.1 Project execution sequence

Run the following commands from the repository root: risk_aware_inspection_prototype/.

Allow the activation script in the current PowerShell session

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

Activate the project Python environment

```
.\.venv\Scripts\Activate.ps1
```

Start and verify PostgreSQL

```
docker compose up -d postgres
```

```
docker compose ps
```

Activate the Anomalib environment for anomaly-detection/backend components

```
.\.venv_anomalib\Scripts\Activate.ps1
```

Start the FastAPI backend

```
uvicorn risk_aware_inspection.api_app:app --app-dir src --reload --port 8000
```

Start the NiceGUI operator dashboard

```
python src\risk_aware_inspection\dashboard_app.py
```

Stop PostgreSQL after use

```
docker compose stop postgres
```

The FastAPI service is available locally at 127.0.0.1:8000, with its automatically generated API documentation at 127.0.0.1:8000/docs. The NiceGUI dashboard is configured to run at 127.0.0.1:8090. Separate terminals can be used for the PostgreSQL service, FastAPI backend and NiceGUI interface so that runtime messages remain visible during validation.

D.2 Repository location and retained project evidence

Repository: <https://github.com/Jayze123/risk-aware-inspection-prototype>

The repository contains the anomaly-detection experiment scripts, localisation evaluation, confidence fusion, semantic interpretation, deterministic risk-priority logic, human-review gating, PostgreSQL audit functions, FastAPI backend, NiceGUI dashboard, configuration files, experiment summaries and project documentation. Git version control was used throughout development to preserve implementation milestones and the dissertation writing artefacts associated with the project.

Reproducibility note: The exact runtime state of the project remains dependent on the locally installed environments, retained model outputs and MVTec AD data paths. The commands above record the operational service sequence used during the completed prototype and should be read together with the configuration and code extracts provided in Appendices A and C.